LAPORAN PRAKTIKUM BIG DATA

MINGGU 5: PIPELINE DATA BERITA DARI INGESTION HINGGA MAPREDUCE



OLEH:

ALFARIS AULIA RAHMAN

2411533006

DOSEN PENGAMPU:

AFDHAL DINILHAK, S.KOM, M.KOM.

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

A. Pendahuluan

Pipeline data berita menghubungkan pengambilan data, pemeriksaan kualitas, penyimpanan, query, dan pemrosesan teks. Praktikum ini menggunakan judul berita detik.com untuk menelusuri aliran data dari portal lokal sampai hasil frekuensi kata yang disimpan sebagai Parquet dan database SQLite.

Dari 91.017 baris sumber diperoleh 90.818 judul bersih. Sembilan partisi kategori dipakai sebagai input split MapReduce. Combiner mengurangi pasangan yang dikirim dari 825.601 menjadi 74.009, sekitar 91,04%, tanpa mengubah hasil. Temuan ini menunjukkan bahwa pengurangan perpindahan data lebih menentukan daripada sekadar menambah proses pada dataset kecil.

Data historis 1 Januari sampai 13 Juni 2020 dipisahkan dari 20 judul terbaru yang diambil melalui sitemap. Eksekusi dilakukan pada mesin lokal; pengujian multiprocessing memakai fork sebagaimana default Linux dan Colab. Karena itu angka waktu merupakan hasil mesin pengujian, bukan ukuran kinerja cluster atau jaminan waktu pada Colab.

B. Tujuan

1. Mengambil judul berita melalui HTML dan sitemap XML dengan memperhatikan robots.txt, jeda, serta batas request.
2. Menyalurkan event melalui source, channel, dan sink ke lapisan data mentah.
3. Memeriksa kualitas, membersihkan data, dan menyimpan Parquet berpartisi.
4. Menjelaskan dan menerapkan map, shuffle, serta reduce untuk frekuensi kata.
5. Mengukur manfaat combiner dan mengidentifikasi ketimpangan beban reducer.
6. Menguji Hadoop Streaming melalui pipe serta memastikan hasilnya sama dengan versi Python.

C. Dasar Teori dan Dataset

1. Ingestion dan lapisan data

Ingestion membawa data dari sumber ke sistem analisis. Crawling mengambil halaman, lalu parser memilih elemen yang diperlukan. Pada pendekatan source-channel-sink, antrean memisahkan kecepatan pengambil data dari penulis berkas. Saat channel penuh, source menunggu sehingga penggunaan memori terkendali.

Lapisan mentah menyimpan event JSONL tanpa pembersihan. Lapisan terkurasi menyimpan hasil cleaning dalam Parquet berpartisi. Lapisan siap pakai memuat hasil

agregasi yang dapat dibaca untuk pelaporan. Pemisahan ini mempertahankan jejak transformasi dari sumber sampai hasil.

2. MapReduce dan biaya shuffle

   Map mengubah setiap split menjadi pasangan key dan value. Shuffle mengelompokkan seluruh pasangan dengan key yang sama, sedangkan reduce menggabungkan nilainya. Pada word count, mapper menghasilkan pasangan kata dan angka satu; reducer menjumlahkan seluruh angka untuk setiap kata.

   Combiner melakukan penjumlahan lokal sebelum shuffle. Partitioner CRC32 menentukan reducer tujuan secara stabil. Satu kata yang sangat sering muncul tetap menuju satu reducer, sehingga menambah reducer belum tentu menghilangkan skew. HDFS dan YARN merupakan bagian ekosistem Hadoop, tetapi tidak dijalankan pada simulasi Python maupun pipe lokal dalam praktikum ini.

3. Dataset dan batas interpretasi

   Dataset historis berasal dari repositori ibamibrahim/dataset-judul-berita-indonesia. CSV berukuran sekitar 16,1 MB memiliki kolom date, url, title, dan category. Dataset hanya memuat judul; hasilnya tidak mewakili isi lengkap artikel. Periode Juni hanya mencakup 13 hari, sehingga tidak langsung setara dengan bulan penuh.

   Sitemap terbaru dipakai untuk latihan crawling dan kata kunci, bukan untuk mengubah angka acuan dataset 2020. Metadata URL inet pada dataset historis berisi tanda # untuk seluruh kategorinya. Teks judul tetap dipertahankan untuk analisis, sementara URL dan ID yang tidak valid dikosongkan.

Tabel 1. Jumlah baris tiap kategori sebelum dan sesudah cleaning

Kategori	Mentah	Bersih
news	32.360	32.321
hot	16.330	16.290
finance	14.168	14.141
travel	6.466	6.453
inet	5.640	5.577
health	4.919	4.914
oto	4.383	4.375

Kategori	Mentah	Bersih
food	4.315	4.314
sport	2.436	2.433
Total	91.017	90.818

D. Langkah Pengerjaan Praktikum

1. H-1 Peta pipeline dan permainan kartu kantin

Pipeline terdiri atas ingestion, quality dan cleaning, storage, query dan modeling, processing, serta serving. Permainan kartu memperagakan map pada tiap kasir, shuffle berdasarkan menu, dan reduce berupa penjumlahan porsi.

Tabel 2. Hasil reduce permainan kantin

Menu	Jumlah porsi
nasi goreng	5
es teh	8
mie ayam	4
Total	17

Hasilnya adalah nasi goreng 5 porsi, es teh 8 porsi, dan mie ayam 4 porsi. Jumlah kartu berpindah berkurang dari 12 menjadi 9 dengan combiner, sedangkan total 17 porsi tetap sama.

2. H-2 Persiapan data dan portal lokal

Pustaka dipasang, CSV diunduh, dan ukuran serta skema data diperiksa. Portal lokal menyajikan 100 judul per halaman melalui alamat 127.0.0.1:8000. Kode berikut mengikuti langkah persiapan modul.

Notebook Praktikum 5 | Sel 3 | Kode | Baris 1–1

```
1 !pip install -q duckdb beautifulsoup4 lxml requests pyarrow pandas
```

Gambar 1. Kode H-2

Notebook Praktikum 5 | Sel 3 | Hasil eksekusi

```
[notice] A new release of pip is available: 26.2.1 -> 26.2.2
[notice] To update, run: pip install --upgrade pip
```

Gambar 2. Hasil H-2

Notebook Praktikum 5 | Sel 6 | Kode | Baris 1–14

```
import os, re, json, time, zlib, queue, threading, collections
from pathlib import Path
import urllib.request
import pandas as pd

URL_DATA = ("https://raw.githubusercontent.com/ibamibrahim/"
            "dataset-judul-berita-indonesia/master/detik_news_title.csv")
if not os.path.exists("detik_news_title.csv"):
    urllib.request.urlretrieve(URL_DATA, "detik_news_title.csv")

df_sumber = pd.read_csv("detik_news_title.csv")
print("Ukuran berkas :", round(os.path.getsize("detik_news_title.csv") /
1e6, 1), "MB")
print("Baris, kolom  :", df_sumber.shape)
print("Kolom         :", list(df_sumber.columns))
```

Gambar 3. Kode H-2

Notebook Praktikum 5 | Sel 6 | Hasil eksekusi

```
Ukuran berkas : 16.1 MB
Baris, kolom  : (91017, 4)
Kolom         : ['date', 'url', 'title', 'category']
```

Gambar 4. Hasil H-2

Notebook Praktikum 5 | Sel 8 | Kode | Bagian 1/4 | Baris 1–19

```
import http.server, urllib.parse, html

HALAMAN_UKURAN = 100          # judul per halaman

def jalankan_portal(df, port=8000):
    """Portal berita lokal (siap pakai): menyajikan judul dalam halaman
HTML."""
    baris = df.to_dict("records")
    n_hal = (len(baris) + HALAMAN_UKURAN - 1) // HALAMAN_UKURAN

    class Handler(http.server.BaseHTTPRequestHandler):
        def log_message(self, *args):          # matikan log
            pass
        def _kirim(self, kode, teks, tipe="text/html; charset=utf-8"):
            data = teks.encode("utf-8")
            self.send_response(kode)
            self.send_header("Content-Type", tipe)
            self.send_header("Content-Length", str(len(data)))
            self.end_headers()
            self.wfile.write(data)
```

Gambar 5. Kode H-2 bagian 1 dari 4

Notebook Praktikum 5 | Sel 8 | Kode | Bagian 2/4 | Baris 20–35

```
        def do_GET(self):
            u = urllib.parse.urlparse(self.path)
            if u.path == "/robots.txt":
                self._kirim(200, "User-agent: *\nDisallow: /arsip-
privat/\n", "text/plain")
            elif u.path == "/arsip-privat/":
                self._kirim(200, "<p>Halaman ini tidak boleh di-crawl.</p>")
            elif u.path == "/berita":
                q = urllib.parse.parse_qs(u.query)
                hal = int(q.get("hal", ["1"])[0])
                if not 1 <= hal <= n_hal:
                    return self._kirim(404, "<p>Halaman tidak ada</p>")
                potong = baris[(hal - 1) * HALAMAN_UKURAN: hal *
HALAMAN_UKURAN]
                isi = []
                for r in potong:
                    iso = pd.to_datetime(r["date"],
format="%m/%d/%Y").strftime("%Y-%m-%d")
                    isi.append(
```

Gambar 6. Kode H-2 bagian 2 dari 4

Notebook Praktikum 5 | Sel 8 | Kode | Bagian 3/4 | Baris 36–50

```
                        '<article class="item">'
                        f'<h2 class="judul"><a href="
{html.escape(r["url"])}">{html.escape(r["title"])}</a></h2>'
                        f'<span class="kategori">
{html.escape(r["category"])}</span>'
                        f'<time datetime="{iso}">{html.escape(r["date"])}
</time>'

                        '</article>')
                self._kirim(200, f"<html><body><h1>Portal Contoh</h1>
{''.join(isi)}"
                                   f'<nav data-total="{n_hal}"></nav></body>
</html>')
            else:
                self._kirim(404, "<p>Tidak ditemukan</p>")

    server = http.server.ThreadingHTTPServer(("127.0.0.1", port), Handler)
    threading.Thread(target=server.serve_forever, daemon=True).start()
    return server, n_hal

```

Gambar 7. Kode H-2 bagian 3 dari 4

Notebook Praktikum 5 | Sel 8 | Kode | Bagian 4/4 | Baris 51–53

```
server, N_HALAMAN = jalankan_portal(df_sumber)
BASE = "http://127.0.0.1:8000"
print("Portal lokal aktif di", BASE, "| jumlah halaman:", N_HALAMAN)
```

Gambar 8. Kode H-2 bagian 4 dari 4

Notebook Praktikum 5 | Sel 8 | Hasil eksekusi

```
Portal lokal aktif di http://127.0.0.1:8000 | jumlah halaman: 911
```

Gambar 9. Hasil H-2

CSV berisi 91.017 baris dan empat kolom. Portal menyediakan 911 halaman. Portal ini memungkinkan crawling seluruh data tanpa mengirim puluhan ribu request ke situs berita asli.

3. H-3 Crawling portal lokal

Crawler membaca robots.txt, menguji halaman yang diizinkan dan dilarang, lalu mem-parsing judul, URL, kategori, serta tanggal. Fungsi crawl_iter mengatur retry dan jeda antarhalaman.

Notebook Praktikum 5 | Sel 10 | Kode | Baris 1–15

```
import requests, urllib.robotparser
from bs4 import BeautifulSoup

UA = "KelasBigData-UNAND/1.0 (praktikum; kontak: dosen)"

def baca_robots(base):
    r = requests.get(base + "/robots.txt", headers={"User-Agent": UA},
timeout=10)
    rp = urllib.robotparser.RobotFileParser()
    rp.parse(r.text.splitlines())
    return rp

robots = baca_robots(BASE)
for jalur in ["/berita?hal=1", "/arsip-privat/"]:
    boleh = robots.can_fetch(UA, BASE + jalur)
    print(f"{jalur:<18} boleh di-crawl? {boleh}")
```

Gambar 10. Kode H-3

Notebook Praktikum 5 | Sel 10 | Hasil eksekusi

```
/berita?hal=1      boleh di-crawl? True
/arsip-privat/     boleh di-crawl? False
```

Gambar 11. Hasil H-3

Notebook Praktikum 5 | Sel 12 | Kode | Bagian 1/3 | Baris 1–17

```python
def parse_halaman(teks_html):
    """Ubah HTML satu halaman menjadi daftar dict (judul, url, kategori, tanggal)."""
    sup = BeautifulSoup(teks_html, "html.parser")
    hasil = []
    for a in sup.select("article.item"):
        link = a.select_one("h2.judul a")
        hasil.append({

            "judul": link.get_text(strip=True),
            "url": link["href"],
            "kategori": a.select_one("span.kategori").get_text(strip=True),
            "tanggal": a.select_one("time")["datetime"],
        })
    return hasil

def crawl_iter(base, halaman, delay=0.0, ua=UA):
    """Siap pakai: ambil halaman satu per satu, patuhi robots.txt, beri jeda."""
```

Gambar 12. Kode H-3 bagian 1 dari 3

Notebook Praktikum 5 | Sel 12 | Kode | Bagian 2/3 | Baris 18–37

```python
    rp = baca_robots(base)
    sesi = requests.Session()
    sesi.headers["User-Agent"] = ua
    for h in halaman:
        url = f"{base}/berita?hal={h}"
        if not rp.can_fetch(ua, url):
            print("Dilewati (robots.txt):", url)
            continue
        for percobaan in range(3):
            try:
                r = sesi.get(url, timeout=10)
                r.raise_for_status()
                break
            except requests.RequestException as e:
                time.sleep(0.5 * (percobaan + 1))
        else:
            print("Gagal setelah 3 percobaan:", url)
            continue
        yield from parse_halaman(r.text)
        time.sleep(delay)
```

Gambar 13. Kode H-3 bagian 2 dari 3

Notebook Praktikum 5 | Sel 12 | Kode | Bagian 3/3 | Baris 38–42

```
38
39 contoh = list(crawl_iter(BASE, range(1, 4)))
40 print("Judul dari 3 halaman:", len(contoh))
41 for c in contoh[:3]:
42     print(c)
```

Gambar 14. Kode H-3 bagian 3 dari 3

Notebook Praktikum 5 | Sel 12 | Hasil eksekusi

```
Judul dari 3 halaman: 300
{'judul': 'Kemnaker Awasi TKA di Meikarta', 'url':
'https://finance.detik.com/berita-ekonomi-bisnis/d-4916114/kemnaker-awasi-tka-
di-meikarta', 'kategori': 'finance', 'tanggal': '2020-02-26'}
{'judul': 'BNI Digitalkan BNI Java Jazz 2020', 'url':
'https://finance.detik.com/berita-ekonomi-bisnis/d-4916199/bni-digitalkan-bni-
java-jazz-2020', 'kategori': 'finance', 'tanggal': '2020-02-26'}
{'judul': 'Terbang ke Australia, Edhy Prabowo Mau Genjot Budi Daya Lobster',
'url': 'https://finance.detik.com/berita-ekonomi-bisnis/d-4916193/terbang-ke-
australia-edhy-prabowo-mau-genjot-budi-daya-lobster', 'kategori': 'finance',
'tanggal': '2020-02-26'}
```

Gambar 15. Hasil H-3

Halaman /berita diizinkan, sedangkan /arsip-privat/ tidak diizinkan. Tiga halaman percobaan menghasilkan 300 judul. Tanggal diambil dari atribut datetime agar formatnya tidak ambigu.

4. H-4 Crawling sitemap berita

Parser XML diuji lebih dahulu pada tiga entri contoh. Setelah itu crawler membaca tiga sitemap berita dengan jeda dua detik, batas tujuh judul per bagian, dan batas total 20 judul. User-Agent menggunakan kontak alfarisauliarahman@gmail.com.

Notebook Praktikum 5 | Sel 14 | Kode | Bagian 1/2 | Baris 1–18

```
from bs4 import BeautifulSoup
from urllib.parse import urlparse

POLA_ARTIKEL = re.compile(r"https?://([\w-]+)\.detik\.com/.+/d-(\d+)/")

def parse_sitemap_news(xml_data):
    """Ubah sitemap berita (XML) menjadi daftar dict (judul, url, kategori, id, tanggal, kata_kunci)."""
    sup = BeautifulSoup(xml_data, "xml")
    hasil = []
    for u in sup.find_all("url"):
        loc, judul, waktu = u.find("loc"), u.find("title"), u.find("publication_date")
        if loc is None or judul is None:
            continue
        url = loc.get_text(strip=True)
        m = POLA_ARTIKEL.match(url)
        kunci = u.find("keywords")
        hasil.append({
            "judul": re.sub(r"\s+", " ", judul.get_text(strip=True)),
```

Gambar 16. Kode H-4 bagian 1 dari 2

Notebook Praktikum 5 | Sel 14 | Kode | Bagian 2/2 | Baris 19–25

```
            "url": url,
            "kategori": urlparse(url).netloc.split(".")[0],   # news, inet, sport, ...
            "id": int(m.group(2)) if m else None,
            "tanggal": waktu.get_text(strip=True)[:10] if waktu else None,
            "kata_kunci": kunci.get_text(strip=True) if kunci else "",
        })
    return hasil
```

Gambar 17. Kode H-4 bagian 2 dari 2

Notebook Praktikum 5 | Sel 16 | Kode | Bagian 1/3 | Baris 1–14

```
# Contoh XML: bentuk disalin dari sitemap berita asli (tiga entri hasil uji dosen, 29 Okt 2026).
CONTOH_XML = """<?xml version="1.0" encoding="UTF-8"?>
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9"
        xmlns:news="http://www.google.com/schemas/sitemap-news/0.9">
  <url>
    <loc><![CDATA[https://news.detik.com/berita/d-8729282/mk-kepala-daerah-terdakwa-politik-tak-bisa-diberhentikan-tanpa-usulan-dprd]]></loc>
    <news:news>

<news:publication><news:name>detikNews</news:name>
<news:language>id</news:language></news:publication>
      <news:publication_date>2026-10-29T12:52:00+00:00</news:publication_date>
      <news:title><![CDATA[MK: Kepala Daerah 'Terdakwa Politik' Tak Bisa Diberhentikan Tanpa Usulan DPRD]]></news:title>
      <news:keywords><![CDATA[mahkamah konstitusi,mk,hukum,kepala daerah]]></news:keywords>
    </news:news>
  </url>
```

Gambar 18. Kode H-4 bagian 1 dari 3

Notebook Praktikum 5 | Sel 16 | Kode | Bagian 2/3 | Baris 15–28

```
  <url>

    <loc><![CDATA[https://news.detik.com/berita/d-8729280/tep-unair-periksa-1-000-sapi-di-manokwari-lewat-posyandu-ternak-keliling]]></loc>
    <news:news>
      <news:publication_date>2026-10-29T12:50:00+00:00</news:publication_date>
      <news:title><![CDATA[TEP UNAIR Periksa 1.000 Sapi di Manokwari Lewat Posyandu Ternak Keliling]]></news:title>
    </news:news>
  </url>
  <url>
    <loc><![CDATA[https://news.detik.com/berita/d-8729264/andre-rosiade-dorong-sedimen-sungai-akibat-banjir-jadi-sumber-pendapatan-padang]]></loc>
    <news:news>
      <news:publication_date>2026-10-29T12:44:00+00:00</news:publication_date>
      <news:title><![CDATA[Andre Rosiade Dorong Sedimen Sungai Akibat Banjir Jadi Sumber Pendapatan Padang]]></news:title>
    </news:news>
```

Gambar 19. Kode H-4 bagian 2 dari 3

Notebook Praktikum 5 | Sel 16 | Kode | Bagian 3/3 | Baris 29–35

```
  </url>
</urlset>"""

for x in parse_sitemap_news(CONTOH_XML):
    print({k: x[k] for k in ("judul", "kategori", "id", "tanggal")})
print("kata kunci entri pertama:", parse_sitemap_news(CONTOH_XML)[0]
["kata_kunci"])
print("Sama bila masukan berupa bytes:",
parse_sitemap_news(CONTOH_XML.encode("utf-8")) ==
parse_sitemap_news(CONTOH_XML))
```

Gambar 20. Kode H-4 bagian 3 dari 3

Notebook Praktikum 5 | Sel 16 | Hasil eksekusi

```
{'judul': "MK: Kepala Daerah 'Terdakwa Politik' Tak Bisa Diberhentikan Tanpa
Usulan DPRD", 'kategori': 'news', 'id': 8729282, 'tanggal': '2026-10-29'}
{'judul': 'TEP UNAIR Periksa 1.000 Sapi di Manokwari Lewat Posyandu Ternak
Keliling', 'kategori': 'news', 'id': 8729280, 'tanggal': '2026-10-29'}
{'judul': 'Andre Rosiade Dorong Sedimen Sungai Akibat Banjir Jadi Sumber
Pendapatan Padang', 'kategori': 'news', 'id': 8729264, 'tanggal': '2026-10-
29'}
kata kunci entri pertama: mahkamah konstitusi,mk,hukum,kepala daerah
Sama bila masukan berupa bytes: True
```

Gambar 21. Hasil H-4

Notebook Praktikum 5 | Sel 18 | Kode | Bagian 1/3 | Baris 1–18

```
import urllib.robotparser

UA_DETIK = "BigData/1.0 (praktikum pembelajaran; kontak:
alfarisauliarahman@gmail.com)"
DELAY = 2.0                 # detik antar-request. Jangan dikurangi.
BATAS_PER_BAGIAN = 7        # judul yang diambil dari tiap sitemap
BATAS_TOTAL = 20            # batas seluruh crawling
SITEMAP_DIPILIH = [
    "https://news.detik.com/berita/sitemap_news.xml",
    "https://inet.detik.com/news/sitemap_news.xml",
    "https://sport.detik.com/basket/sitemap_news.xml",
]

_robots = {}
def boleh_diambil(url, ua=UA_DETIK):
    """robots.txt berlaku per host: news.detik.com dan inet.detik.com punya
berkas sendiri."""
    host = urlparse(url).netloc
    if host not in _robots:
        rp = urllib.robotparser.RobotFileParser()
```

Gambar 22. Kode H-4 bagian 1 dari 3

Notebook Praktikum 5 | Sel 18 | Kode | Bagian 2/3 | Baris 19–36

```
        rp.set_url(f"https://{host}/robots.txt")
        rp.read()
        _robots[host] = rp
    return _robots[host].can_fetch(ua, url)


def ambil_xml(url, sesi, delay=DELAY):
    if not boleh_diambil(url):
        raise PermissionError("robots.txt melarang: " + url)
    r = sesi.get(url, timeout=20)
    if r.status_code in (403, 429):
        raise RuntimeError(f"Server membalas {r.status_code}. Berhenti,
tunggu, dan kurangi laju.")
    r.raise_for_status()
    time.sleep(delay)
    return r.content            # bytes: pengkodean dibaca dari deklarasi
XML

def crawl_detik():
    sesi = requests.Session()
```

Gambar 23. Kode H-4 bagian 2 dari 3

Notebook Praktikum 5 | Sel 18 | Kode | Bagian 3/3 | Baris 37–43

```
    sesi.headers["User-Agent"] = UA_DETIK
    hasil = []
    for sm in SITEMAP_DIPILIH:
        if len(hasil) >= BATAS_TOTAL:
            break
        hasil += parse_sitemap_news(ambil_xml(sm, sesi))[:BATAS_PER_BAGIAN]
    return hasil[:BATAS_TOTAL]
```

Gambar 24. Kode H-4 bagian 3 dari 3

Notebook Praktikum 5 | Sel 20 | Kode | Baris 1–18

```
try:
    sampel = crawl_detik()
except Exception as e:
    sampel = []
    print("Crawling gagal:", type(e).__name__, "-", e)

if sampel:
    Path("contoh_crawl").mkdir(exist_ok=True)
    with open("contoh_crawl/detik_sitemap.jsonl", "w", encoding="utf-8") as
f:
        for x in sampel:
            f.write(json.dumps(x, ensure_ascii=False) + "\n")
    cek = pd.DataFrame(sampel)
    print("Judul terkumpul   :", len(cek))
    print("Judul kosong      :", (cek["judul"].str.strip() == "").sum())
    print("URL duplikat      :", cek["url"].duplicated().sum())
    print("Tanggal kosong    :", cek["tanggal"].isna().sum())
    print("Kategori          :", cek["kategori"].value_counts().to_dict())
    print(cek[["judul", "kategori",
"tanggal"]].head(3).to_string(index=False))
```

Gambar 25. Kode H-4

Notebook Praktikum 5 | Sel 20 | Hasil eksekusi

```
Judul terkumpul   : 20
Judul kosong      : 0
URL duplikat      : 0
Tanggal kosong    : 0
Kategori          : {'news': 7, 'inet': 7, 'sport': 6}

judul kategori    tanggal
                                    Polri Terbitkan SE Larangan Personel
Ngevape     news 2026-11-01
            Sopir Truk Molen Picu Kecelakaan Tewaskan 6 Orang di Bekasi
Ditangkap     news 2026-11-01
IPW Dukung Polisi Usut Pembakaran Warung Mie di Sukoharjo, Ingatkan
Akuntabilitas     news 2026-11-01
```

Gambar 26. Hasil H-4

Pengambilan langsung menghasilkan 20 judul: news 7, inet 7, dan sport 6. Tidak ada judul kosong, URL duplikat, atau tanggal kosong. Hasil terbaru disimpan terpisah di contoh_crawl sehingga tidak memengaruhi angka dataset historis.

5. H-5 Channel ala Flume

Seluruh halaman portal lokal diambil sebagai source, queue.Queue berkapasitas 2.000 event dipakai sebagai channel, dan sink menulis batch 5.000 event menjadi JSONL.

Notebook Praktikum 5 | Sel 22 | Kode | Bagian 1/3 | Baris 1–18

```
RAW = Path("lapisan_mentah"); RAW.mkdir(exist_ok=True)
for f in RAW.glob("*.jsonl"):
    f.unlink()

channel = queue.Queue(maxsize=2000)
STOP = object()
statistik = {"max_antrean": 0, "masuk": 0, "keluar": 0, "file": 0}

def source():
    for ev in crawl_iter(BASE, range(1, N_HALAMAN + 1)):
        channel.put(ev)                          # menunggu bila channel penuh
        statistik["masuk"] += 1
        statistik["max_antrean"] = max(statistik["max_antrean"],
channel.qsize())
    channel.put(STOP)

def sink(ukuran_batch=5000):
    batch, nomor = [], 0
    def tulis():
```

Gambar 27. Kode H-5 bagian 1 dari 3

Notebook Praktikum 5 | Sel 22 | Kode | Bagian 2/3 | Baris 19–36

```
        nonlocal batch, nomor
        nomor += 1
        with open(RAW / f"batch_{nomor:04d}.jsonl", "w", encoding="utf-8")
as f:
            for ev in batch:
                f.write(json.dumps(ev, ensure_ascii=False) + "\n")
        statistik["file"] = nomor
        batch = []
    while True:
        ev = channel.get()
        if ev is STOP:
            break
        batch.append(ev); statistik["keluar"] += 1
        if len(batch) >= ukuran_batch:
            tulis()
    if batch:
        tulis()

t0 = time.perf_counter()
```

Gambar 28. Kode H-5 bagian 2 dari 3

Notebook Praktikum 5 | Sel 22 | Kode | Bagian 3/3 | Baris 37–47

```
t_src = threading.Thread(target=source); t_snk =
threading.Thread(target=sink)
t_src.start(); t_snk.start(); t_src.join(); t_snk.join()
waktu = time.perf_counter() - t0
print("Event masuk channel :", f"{statistik['masuk']:,}")
print("Event ditulis sink  :", f"{statistik['keluar']:,}")
print("File batch          :", statistik["file"], "->", sorted(p.name for p
in RAW.glob('*.jsonl'))[:3],
"...")
print("Antrean maks <= 2000:", statistik["max_antrean"] <= 2000)

print("Total ukuran mentah :", round(sum(p.stat().st_size for p in
RAW.glob('*.jsonl')) / 1e6, 1), "MB")
print("Waktu crawl total   :", round(waktu, 1), "detik untuk", N_HALAMAN,
"halaman")
```

Gambar 29. Kode H-5 bagian 3 dari 3

Notebook Praktikum 5 | Sel 22 | Hasil eksekusi

```
Event masuk channel : 91,017
Event ditulis sink  : 91,017
File batch          : 19 -> ['batch_0001.jsonl', 'batch_0002.jsonl',
'batch_0003.jsonl'] ...
Antrean maks <= 2000: True
Total ukuran mentah : 20.7 MB
Waktu crawl total   : 7.6 detik untuk 911 halaman
```

Gambar 30. Hasil H-5

Jumlah event masuk dan ditulis sama, yaitu 91.017. Terbentuk 19 berkas dengan ukuran total sekitar 20,7 MB. Waktu crawl pada pengujian terbaru 7,6 detik untuk 911 halaman; ukuran antrean tidak melampaui batas.

6. H-6 Quality cleaning dan Parquet berpartisi

Pemeriksaan dilakukan sebelum pembersihan: judul kosong, panjang judul, kategori dikenal, validitas tanggal, pola URL artikel, serta judul duplikat. Cleaning merapikan spasi, menyimpan salinan judul pertama, membatasi panjang, dan membentuk bulan serta ID.

Notebook Praktikum 5 | Sel 24 | Kode | Bagian 1/2 | Baris 1–15

```
mentah = pd.concat([pd.read_json(p, lines=True) for p in
sorted(RAW.glob("*.jsonl"))],
                   ignore_index=True)
print("Baris mentah:", f"{len(mentah):,}")

KATEGORI_DIKENAL = {"finance", "food", "health", "hot", "inet", "news",
"oto", "sport", "travel"}

def aturan_kualitas(d):
    """Setiap aturan menghasilkan Series True/False per baris."""
    jml_kata = d["judul"].fillna("").str.split().str.len()
    return {
        "judul tidak kosong"      : d["judul"].fillna("").str.strip() != "",
        "judul 3–30 kata"         : jml_kata.between(3, 30),
        "kategori dikenal"        : d["kategori"].isin(KATEGORI_DIKENAL),
        "tanggal valid"           : pd.to_datetime(d["tanggal"], format="%Y-
%m-%d", errors="coerce").notna(),
        "url artikel valid"       : d["url"].fillna("").str.contains(r"/d-
\d+", regex=True),
```

Gambar 31. Kode H-6 bagian 1 dari 2

Notebook Praktikum 5 | Sel 24 | Kode | Bagian 2/2 | Baris 16–20

```
        "judul tidak duplikat"    : ~d["judul"].duplicated(keep=False),
    }

for nama, lulus in aturan_kualitas(mentah).items():
    print(f"{nama:<24} gagal: {(~lulus).sum():>6,}
({(~lulus).mean()*100:5.2f}%)")
```

Gambar 32. Kode H-6 bagian 2 dari 2

Notebook Praktikum 5 | Sel 24 | Hasil eksekusi

```
Baris mentah: 91,017
judul tidak kosong       gagal:      0  ( 0.00%)
judul 3–30 kata          gagal:     78  ( 0.09%)
kategori dikenal         gagal:      0  ( 0.00%)
tanggal valid            gagal:      0  ( 0.00%)
url artikel valid        gagal:  5,640  ( 6.20%)
judul tidak duplikat     gagal:    235  ( 0.26%)
```

Gambar 33. Hasil H-6

Notebook Praktikum 5 | Sel 26 | Kode | Baris 1–9

```
bersih = mentah.copy()
bersih["judul"] = bersih["judul"].str.replace(r"\s+", " ",
regex=True).str.strip()
bersih = bersih.drop_duplicates(subset="judul", keep="first")
bersih = bersih[bersih["judul"].str.split().str.len().between(3, 30)].copy()
bersih["tanggal"] = pd.to_datetime(bersih["tanggal"], format="%Y-%m-%d")
bersih["bulan"] = bersih["tanggal"].dt.strftime("%Y-%m")
bersih["id"] = bersih["url"].str.extract(r"/d-(\d+)")[0].astype("Int64")   #
kosong bila url rusak
bersih["url"] = bersih["url"].where(bersih["url"].str.contains(r"/d-\d+",
regex=True))
print("Baris setelah dibersihkan:", f"{len(bersih):,}", "| dibuang:", f"
{len(mentah) - len(bersih):,}")
```

Gambar 34. Kode H-6

Notebook Praktikum 5 | Sel 26 | Hasil eksekusi

```
Baris setelah dibersihkan: 90,818 | dibuang: 199
```

Gambar 35. Hasil H-6

Notebook Praktikum 5 | Sel 28 | Kode | Bagian 1/2 | Baris 1–18

```
import shutil, pyarrow.dataset as ds

def simpan_terpartisi(data, kolom_partisi, folder):
    shutil.rmtree(folder, ignore_errors=True)
    data.to_parquet(folder, partition_cols=kolom_partisi, index=False)
    n = len(list(Path(folder).rglob("*.parquet")))
    ukuran = sum(p.stat().st_size for p in Path(folder).rglob("*.parquet"))
/ 1e6
    return n, ukuran

# BAGIAN KUNCI: bandingkan dua pilihan kolom partisi
for kolom in (["kategori"], ["bulan"], ["kategori", "bulan"]):
    n, mb = simpan_terpartisi(bersih, kolom, "uji_partisi")
    print(f"partisi {str(kolom):<22} -> {n:>3} file, {mb:5.1f} MB")
shutil.rmtree("uji_partisi")

TERKURASI = "lapisan_terkurasi"
n, mb = simpan_terpartisi(bersih, ["kategori"], TERKURASI)
print("\nDisimpan ke", TERKURASI, "dengan partisi kategori:", n, "file,",
round(mb, 1), "MB")
```

Gambar 36. Kode H-6 bagian 1 dari 2

Notebook Praktikum 5 | Sel 28 | Kode | Bagian 2/2 | Baris 19–23

```
19
20 dataset = ds.dataset(TERKURASI, format="parquet", partitioning="hive")
21 semua = list(dataset.get_fragments())
22 hanya = list(dataset.get_fragments(filter=ds.field("kategori") ==
   "finance"))
23 print("Partition pruning: baca", len(hanya), "dari", len(semua), "file untuk
   kategori = finance")
```

Gambar 37. Kode H-6 bagian 2 dari 2

Notebook Praktikum 5 | Sel 28 | Hasil eksekusi

```
partisi ['kategori']           ->   9 file,   8.4 MB
partisi ['bulan']              ->   6 file,   8.3 MB
partisi ['kategori', 'bulan']  ->  54 file,   8.4 MB

Disimpan ke lapisan_terkurasi dengan partisi kategori: 9 file, 8.4 MB
Partition pruning: baca 1 dari 9 file untuk kategori = finance
```

Gambar 38. Hasil H-6

Terdapat 78 judul di luar 3-30 kata, 235 baris yang termasuk kelompok judul duplikat, dan 5.640 URL tidak valid. Setelah cleaning tersisa 90.818 baris atau berkurang 199. Angka gagal antaraturan tidak dijumlahkan karena satu baris dapat gagal lebih dari satu aturan. Partisi kategori menghasilkan 9 file; bulan 6 file; kategori dan bulan 54 file. Query finance dapat membaca 1 dari 9 file.

7. H-7 Query DuckDB dan bentuk dokumen

DuckDB membaca Parquet melalui hive partitioning. Query menghitung jumlah judul per kategori dan judul yang memuat corona per bulan. Satu record kemudian ditampilkan sebagai dokumen dengan daftar kata.

Notebook Praktikum 5 | Sel 30 | Kode | Bagian 1/2 | Baris 1–18

```
import duckdb
con = duckdb.connect()
con.sql(f"CREATE VIEW berita AS SELECT * FROM
read_parquet('{TERKURASI}/*/*.parquet', hive_partitioning=true)")

print(con.sql("SELECT kategori, COUNT(*) AS jumlah FROM berita GROUP BY
kategori ORDER BY jumlah DESC").df().to_string(index=False))
print()
print(con.sql("""
    SELECT bulan, COUNT(*) AS judul_corona
    FROM berita WHERE lower(judul) LIKE '%corona%'
    GROUP BY bulan ORDER BY bulan""").df().to_string(index=False))

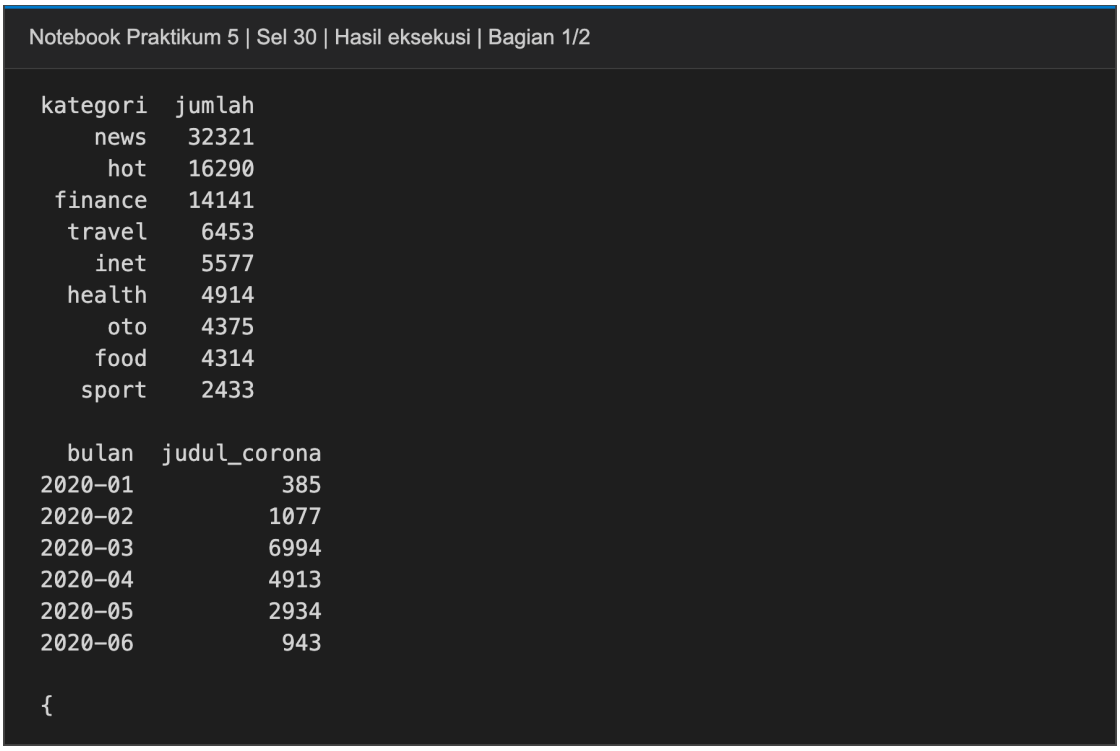
dokumen = {
    "id": int(bersih.iloc[0]["id"]),
    "judul": bersih.iloc[0]["judul"],
    "kategori": bersih.iloc[0]["kategori"],
    "tanggal": str(bersih.iloc[0]["tanggal"].date()),
    "kata": re.findall(r"[a-z]+", bersih.iloc[0]["judul"].lower()),
}
```

Gambar 39. Kode H-7 bagian 1 dari 2

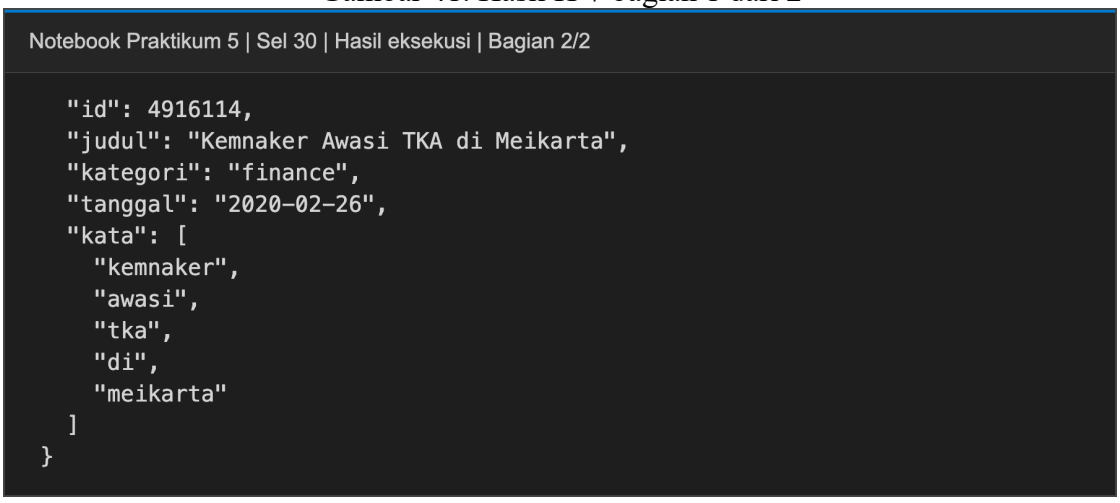
Notebook Praktikum 5 | Sel 30 | Kode | Bagian 2/2 | Baris 19–20

```
print()
print(json.dumps(dokumen, ensure_ascii=False, indent=2))
```

Gambar 40. Kode H-7 bagian 2 dari 2



Gambar 41. Hasil H-7 bagian 1 dari 2



Gambar 42. Hasil H-7 bagian 2 dari 2

Kategori news terbesar, yaitu 32.321 judul. Jumlah judul yang memuat corona memuncak pada Maret sebanyak 6.994. Bentuk tabel memudahkan agregasi, sedangkan dokumen menyatukan satu berita dengan atribut turunannya.

8. H-8 Mini MapReduce kasus kantin

Fungsi map_fn menghasilkan pasangan menu-jumlah, shuffle mengelompokkan nilai per menu, dan reduce_fn menjumlahkannya. Combiner melakukan penjumlahan di masing-masing kasir sebelum pengiriman.

Notebook Praktikum 5 | Sel 32 | Kode | Bagian 1/3 | Baris 1–17

```python
kasir = [
    ["nasi goreng,1", "es teh,2", "mie ayam,1", "es teh,1"],      # kasir A
    ["nasi goreng,2", "es teh,1", "es teh,3", "mie ayam,1"],      # kasir B
    ["mie ayam,2", "nasi goreng,1", "es teh,1", "nasi goreng,1"], # kasir C
]

def map_fn(baris):
    menu, jumlah = baris.split(",")
    return (menu, int(jumlah))

def reduce_fn(menu, daftar_nilai):
    return (menu, sum(daftar_nilai))

# 1) MAP: setiap kasir memproses datanya sendiri
hasil_map = [[map_fn(b) for b in k] for k in kasir]
for nama, h in zip("ABC", hasil_map):
    print("map kasir", nama, "->", h)
```

Gambar 43. Kode H-8 bagian 1 dari 3

Notebook Praktikum 5 | Sel 32 | Kode | Bagian 2/3 | Baris 18–37

```python

# 2) SHUFFLE: kumpulkan nilai berdasarkan key
shuffle = {}
for h in hasil_map:
    for menu, jumlah in h:
        shuffle.setdefault(menu, []).append(jumlah)
print("\nshuffle:", shuffle)

# 3) REDUCE: gabungkan nilai per key
hasil = dict(reduce_fn(m, v) for m, v in shuffle.items())
print("\nreduce:", hasil, "| total porsi:", sum(hasil.values()))

# Combiner: jumlahkan dulu di setiap kasir sebelum dikirim
def combiner(pasangan):
    c = collections.Counter()
    for menu, jumlah in pasangan:
        c[menu] += jumlah
    return list(c.items())

setelah = [combiner(h) for h in hasil_map]
```

Gambar 44. Kode H-8 bagian 2 dari 3

Notebook Praktikum 5 | Sel 32 | Kode | Bagian 3/3 | Baris 38–39

```python
print("\nPasangan dikirim tanpa combiner:", sum(len(h) for h in hasil_map))
print("Pasangan dikirim dengan combiner:", sum(len(h) for h in setelah))
```

Gambar 45. Kode H-8 bagian 3 dari 3

Notebook Praktikum 5 | Sel 32 | Hasil eksekusi

```
map kasir A -> [('nasi goreng', 1), ('es teh', 2), ('mie ayam', 1), ('es teh',
1)]
map kasir B -> [('nasi goreng', 2), ('es teh', 1), ('es teh', 3), ('mie ayam',
1)]
map kasir C -> [('mie ayam', 2), ('nasi goreng', 1), ('es teh', 1), ('nasi
goreng', 1)]

shuffle: {'nasi goreng': [1, 2, 1, 1], 'es teh': [2, 1, 1, 3, 1], 'mie ayam':
[1, 1, 2]}

reduce: {'nasi goreng': 5, 'es teh': 8, 'mie ayam': 4} | total porsi: 17

Pasangan dikirim tanpa combiner: 12
Pasangan dikirim dengan combiner: 9
```

Gambar 46. Hasil H-8

Kode membuktikan hasil permainan H-1: nasi goreng 5, es teh 8, mie ayam 4. Pengiriman berkurang dari 12 pasangan menjadi 9 tanpa mengubah hasil.

9. H-9 Word count dengan partisi sebagai input split

Setiap file partisi menjadi satu input split. Mapper mengeluarkan satu pasangan untuk setiap token huruf a-z. Semua pasangan dikumpulkan berdasarkan kata, lalu frekuensinya dijumlahkan dan dicek terhadap Counter biasa.

Notebook Praktikum 5 | Sel 34 | Kode | Bagian 1/3 | Baris 1–16

```
1  STOPWORD = set("""yang di dan ke dari ini itu untuk dengan pada adalah akan
   tak tidak jadi soal
2  saat ada bisa juga oleh atau karena sebagai dalam usai hingga jika saja para
   lebih hanya
3  sudah belum masih kini lagi pun serta agar bakal dua tiga kok nih ya yg
   dgn""".split())
4
5  def tokenisasi(teks):
6      return re.findall(r"[a-z]+", teks.lower())
7
8  # Input split = satu file partisi Parquet (satu kategori)
9  splits = sorted(Path(TERKURASI).rglob("*.parquet"))
10 print("Jumlah input split:", len(splits))
11 for p in splits:
12     n = len(pd.read_parquet(p, columns=["judul"]))
13     print(f"  {p.parent.name:<16} {n:>6,} judul")
14
15 def mapper_split(path):
16     """Satu mapper memproses satu split dan mengembalikan pasangan (kata,
   1)."""
```

Gambar 47. Kode H-9 bagian 1 dari 3

Notebook Praktikum 5 | Sel 34 | Kode | Bagian 2/3 | Baris 17–35

```
    judul = pd.read_parquet(path, columns=["judul"])["judul"]
    return [(kata, 1) for t in judul for kata in tokenisasi(t)]

hasil_map = [mapper_split(p) for p in splits]
total_pasangan = sum(len(h) for h in hasil_map)

shuffle = collections.defaultdict(list)
for h in hasil_map:
    for kata, n in h:
        shuffle[kata].append(n)
hasil_semua = {k: sum(v) for k, v in shuffle.items()}

print("\nPasangan dari semua mapper:", f"{total_pasangan:,}")
print("Kata unik (key setelah shuffle):", f"{len(hasil_semua):,}")
print("\n10 kata teratas (semua kata):")
for k, n in collections.Counter(hasil_semua).most_common(10):
    print(f"  {k:<10} {n:>7,}")
top = [(k, n) for k, n in collections.Counter(hasil_semua).most_common() if
k not in STOPWORD]
print("\n10 kata teratas (tanpa stopword):")
```

Gambar 48. Kode H-9 bagian 2 dari 3

Notebook Praktikum 5 | Sel 34 | Kode | Bagian 3/3 | Baris 36–41

```
for k, n in top[:10]:
    print(f"  {k:<10} {n:>7,}")

# Cek silang dengan cara biasa (tanpa MapReduce)
cek = collections.Counter(kata for t in bersih["judul"] for kata in
tokenisasi(t))
print("\nSama dengan Counter biasa:", dict(cek) == hasil_semua)
```

Gambar 49. Kode H-9 bagian 3 dari 3

Notebook Praktikum 5 | Sel 34 | Hasil eksekusi | Bagian 1/2

```
Jumlah input split: 9
  kategori=finance 14,141 judul
  kategori=food     4,314 judul
  kategori=health   4,914 judul
  kategori=hot     16,290 judul
  kategori=inet     5,577 judul
  kategori=news    32,321 judul
  kategori=oto      4,375 judul
  kategori=sport    2,433 judul
  kategori=travel   6,453 judul

Pasangan dari semua mapper: 825,601
Kata unik (key setelah shuffle): 32,275

10 kata teratas (semua kata):
  di          27,390
  corona      17,239
  ini         10,124
  yang         8,668
  dan          6,785
```

Gambar 50. Hasil H-9 bagian 1 dari 2

Notebook Praktikum 5 | Sel 34 | Hasil eksekusi | Bagian 2/2

```
  ke           5,592
  tak          5,096
  dari         4,697
  jadi         4,486
  untuk        3,723

10 kata teratas (tanpa stopword):
  corona      17,239
  rp           3,154
  virus        3,075
  kasus        2,743
  polisi       2,732
  positif      2,682
  indonesia    2,504
  warga        2,495
  rumah        2,465
  jokowi       2,317

Sama dengan Counter biasa: True
```

Gambar 51. Hasil H-9 bagian 2 dari 2

Sembilan input split menghasilkan 825.601 pasangan dan 32.275 kata unik. Kata di muncul 27.390 kali; corona 17.239 kali. Hasil MapReduce sama dengan Counter biasa. Daftar stopword hanya digunakan saat menampilkan daftar kata teratas tanpa kata fungsi, bukan untuk mengubah total semua token.

10. H-10 Combiner dan perpindahan data

Pengujian membandingkan satu dan dua proses, masing-masing tanpa dan dengan combiner. Waktu yang dicatat merupakan median tiga kali eksekusi. Combiner memakai Counter di mapper untuk meringkas pasangan sebelum dikembalikan.

Notebook Praktikum 5 | Sel 36 | Kode | Bagian 1/2 | Baris 1–20

```
from multiprocessing import Pool

def mapper_combiner(path):
    """Mapper + combiner: hitung dulu di sisi mapper."""
    judul = pd.read_parquet(path, columns=["judul"])["judul"]
    c = collections.Counter()
    for t in judul:
        c.update(tokenisasi(t))
    return c

def jalankan(fn, daftar, proses):
    t0 = time.perf_counter()
    if proses == 1:
        hasil = [fn(x) for x in daftar]
    else:
        with Pool(proses) as pool:
            hasil = pool.map(fn, daftar)
    return hasil, time.perf_counter() - t0

print("CPU tersedia:", os.cpu_count())
```

Gambar 52. Kode H-10 bagian 1 dari 2

Notebook Praktikum 5 | Sel 36 | Kode | Bagian 2/2 | Baris 21–36

```
print(f"{'skenario':<28}{'waktu (s)':>10}{'pasangan dikirim':>20}")

for proses in (1, 2):
    waktu = []
    for _ in range(3):
        h, w = jalankan(mapper_split, splits, proses); waktu.append(w)
    print(f"{proses} proses, tanpa combiner".ljust(28), f"{sorted(waktu)
[1]:>10.3f}", f"{sum(len(x) for x in h):>20,}")
    waktu = []
    for _ in range(3):
        h2, w = jalankan(mapper_combiner, splits, proses); waktu.append(w)
    print(f"{proses} proses, dengan combiner".ljust(28), f"{sorted(waktu)
[1]:>10.3f}", f"{sum(len(x) for x in h2):>20,}")

total = collections.Counter()
for c in h2:
    total.update(c)
print("\nHasil dengan combiner sama persis:", dict(total) == hasil_semua)
```

Gambar 53. Kode H-10 bagian 2 dari 2

Notebook Praktikum 5 | Sel 36 | Hasil eksekusi

```
CPU tersedia: 10
skenario                    waktu (s)    pasangan dikirim
1 proses, tanpa combiner          0.116             825,601
1 proses, dengan combiner         0.137              74,009
2 proses, tanpa combiner          0.207             825,601
2 proses, dengan combiner         0.118              74,009

Hasil dengan combiner sama persis: True
```

Gambar 54. Hasil H-10

Tanpa combiner, satu proses membutuhkan 0,116 detik dan dua proses 0,207 detik. Dengan combiner, satu proses 0,137 detik dan dua proses 0,118 detik. Dibanding dua proses tanpa combiner, penggunaan combiner mempercepat skenario dua proses. Namun, dua proses dengan combiner masih sedikit lebih lambat daripada satu proses tanpa combiner pada pengukuran ini. Jumlah pasangan turun menjadi 74.009 dan hasil akhirnya tetap identik.

11. H-11 Data skew dan beban reducer

CRC32 membagi key ke reducer secara deterministik. Beban dihitung dari jumlah pasangan yang diterima, kemudian dibandingkan sebelum dan sesudah combiner serta pada jumlah reducer berbeda.

Notebook Praktikum 5 | Sel 38 | Kode | Bagian 1/2 | Baris 1–19

```python
R = 4        # jumlah reducer

def partitioner(kata, r=R):
    return zlib.crc32(kata.encode()) % r

def beban(pasangan_per_mapper, r=R):
    b = [0] * r
    for h in pasangan_per_mapper:
        for kata, n in h:
            b[partitioner(kata, r)] += 1
    return b

def tampil(nama, b):
    tot = sum(b)
    print(f"{nama:<22}", " ".join(f"R{i}={x:>7,} ({x/tot*100:4.1f}%)" for i, x in enumerate(b)))

tampil("tanpa combiner", beban(hasil_map))
tampil("dengan combiner", beban([list(c.items()) for c in h2]))

```

Gambar 55. Kode H-11 bagian 1 dari 2

Notebook Praktikum 5 | Sel 38 | Kode | Bagian 2/2 | Baris 20–29

```
20
21 print("\nBeban reducer terberat dibanding bagian yang adil (tanpa
   combiner):")
22 for r in (4, 16, 64):
23     b = beban(hasil_map, r)
24     print(f"  {r:>3} reducer: terberat {max(b)/sum(b)*100:5.2f}% | bagian
   adil {100/r:5.2f}%")
25
26 print("\nBeban satu key (tanpa combiner):")
27 tot = sum(hasil_semua.values())
28 for kata, n in collections.Counter(hasil_semua).most_common(5):
29     print(f"  {kata:<8} {n:>7,} pasangan = {n/tot*100:4.1f}% dari total, di
   reducer R{partitioner(kata)}")
```

Gambar 56. Kode H-11 bagian 2 dari 2

Notebook Praktikum 5 | Sel 38 | Hasil eksekusi

```
tanpa combiner         R0=240,836 (29.2%) R1=170,066 (20.6%) R2=190,727
(23.1%) R3=223,972 (27.1%)
dengan combiner        R0= 18,301 (24.7%) R1= 18,267 (24.7%) R2= 18,530
(25.0%) R3= 18,911 (25.6%)

Beban reducer terberat dibanding bagian yang adil (tanpa combiner):
    4 reducer: terberat 29.17% | bagian adil 25.00%
   16 reducer: terberat  8.82% | bagian adil  6.25%
   64 reducer: terberat  4.28% | bagian adil  1.56%

Beban satu key (tanpa combiner):
  di        27,390 pasangan =  3.3% dari total, di reducer R0
  corona    17,239 pasangan =  2.1% dari total, di reducer R2
  ini       10,124 pasangan =  1.2% dari total, di reducer R0
  yang       8,668 pasangan =  1.0% dari total, di reducer R3
  dan        6,785 pasangan =  0.8% dari total, di reducer R3
```

Gambar 57. Hasil H-11

Pada empat reducer, beban terberat tanpa combiner 29,17%, melebihi bagian adil 25%. Pada 64 reducer, beban terberat 4,28%, sedangkan bagian adil hanya 1,56%. Combiner membuat pembagian empat reducer mendekati rata, sekitar 24,7-25,6%.

12. H-12 Hadoop Streaming melalui pipe

Masukan kategori dan judul ditulis ke TSV. Mapper serta reducer membaca stdin dan menulis stdout. Urutan cat, mapper, sort, dan reducer mensimulasikan Hadoop Streaming; sort menjalankan fungsi shuffle.

Notebook Praktikum 5 | Sel 40 | Kode | Bagian 1/2 | Baris 1–19

```
Path("masukan.tsv").write_text(
    "\n".join(f"{k}\t{j}" for k, j in zip(bersih["kategori"],
bersih["judul"])) + "\n",
    encoding="utf-8")

Path("mapper.py").write_text('''import sys, re
for baris in sys.stdin:
    kategori, judul = baris.rstrip("\\n").split("\\t", 1)
    for kata in re.findall(r"[a-z]+", judul.lower()):
        print(f"{kata}\\t1")
''')

Path("reducer.py").write_text('''import sys
kunci_sekarang, total = None, 0
for baris in sys.stdin:
    kunci, nilai = baris.rstrip("\\n").split("\\t")
    if kunci != kunci_sekarang:
        if kunci_sekarang is not None:

            print(f"{kunci_sekarang}\\t{total}")
```

Gambar 58. Kode H-12 bagian 1 dari 2

Notebook Praktikum 5 | Sel 40 | Kode | Bagian 2/2 | Baris 20–25

```
        kunci_sekarang, total = kunci, 0
    total += int(nilai)
if kunci_sekarang is not None:
    print(f"{kunci_sekarang}\\t{total}")
''')
print("Baris masukan:", sum(1 for _ in open("masukan.tsv", encoding="utf-
8")))
```

Gambar 59. Kode H-12 bagian 2 dari 2

Notebook Praktikum 5 | Sel 40 | Hasil eksekusi

```
Baris masukan: 90818
```

Gambar 60. Hasil H-12

Notebook Praktikum 5 | Sel 42 | Kode | Baris 1–5

```
!head -n 3 masukan.tsv | python3 mapper.py

!time (cat masukan.tsv | python3 mapper.py | LC_ALL=C sort | python3
reducer.py > hasil_pipe.txt)
!wc -l hasil_pipe.txt
!sort -t$'\t' -k2,2nr hasil_pipe.txt | head -n 5
```

Gambar 61. Kode H-12

Notebook Praktikum 5 | Sel 42 | Hasil eksekusi | Bagian 1/2

```
kemnaker    1
awasi   1
tka 1
di  1
meikarta    1
bni 1
digitalkan  1
bni 1
java    1
jazz    1
terbang 1
ke  1
australia   1
edhy    1
prabowo 1
mau 1
genjot  1
budi    1
daya    1
lobster 1
```

Gambar 62. Hasil H-12 bagian 1 dari 2

Notebook Praktikum 5 | Sel 42 | Hasil eksekusi | Bagian 2/2

```
( cat masukan.tsv | python3 mapper.py | LC_ALL=C sort | python3 reducer.py > )
0.53s user 0.04s system 122% cpu 0.464 total
   32275 hasil_pipe.txt
di  27390
corona  17239
ini 10124
yang    8668
dan 6785
sort: Broken pipe
```

Gambar 63. Hasil H-12 bagian 2 dari 2

Notebook Praktikum 5 | Sel 44 | Kode | Baris 1–5

```
1 hasil_pipe = {}
2 for baris in open("hasil_pipe.txt", encoding="utf-8"):
3     kata, n = baris.rstrip("\n").split("\t")
4     hasil_pipe[kata] = int(n)
5 print("Hasil pipe sama dengan hasil H-9:", hasil_pipe == hasil_semua)
```

Gambar 64. Kode H-12

Notebook Praktikum 5 | Sel 44 | Hasil eksekusi

```
Hasil pipe sama dengan hasil H-9: True
```

Gambar 65. Hasil H-12

Hasil pipe memiliki 32.275 key dan sama persis dengan H-9. Pesan sort: Broken pipe pada pratinjau lima baris terjadi karena head berhenti membaca setelah lima baris, bukan karena perhitungan hasil_pipe gagal. Pemeriksaan kamus hasil tetap True. Instalasi Hadoop asli bersifat opsional dan tidak dijalankan; pengujian ini tidak menggunakan HDFS atau YARN.

13. H-13 Kata teratas per kategori dan ekspor

Key diubah menjadi pasangan kategori-kata. Stopword dibuang di mapper, combiner menghitung lokal, dan reducer menggabungkan seluruh split. Hasilnya disimpan sebagai Parquet dan tabel kata_per_kategori di SQLite sebagai simulasi arah Sqoop export.

Notebook Praktikum 5 | Sel 48 | Kode | Bagian 1/2 | Baris 1–20

```
def mapper_kategori(path):
    """Key gabungan (kategori, kata); combiner dipasang di mapper."""
    df_ = pd.read_parquet(path, columns=["judul"])
    kategori = path.parent.name.split("=")[1]
    c = collections.Counter()
    for t in df_["judul"]:
        for kata in tokenisasi(t):
            if kata not in STOPWORD:
                c[(kategori, kata)] += 1
    return c

gabung = collections.Counter()
for c in map(mapper_kategori, splits):
    gabung.update(c)

per_kategori = collections.defaultdict(list)
for (kat, kata), n in gabung.items():
    per_kategori[kat].append((n, kata))
for kat in sorted(per_kategori):
    top5 = sorted(per_kategori[kat], reverse=True)[:5]
```

Gambar 66. Kode H-13 bagian 1 dari 2

Notebook Praktikum 5 | Sel 48 | Kode | Bagian 2/2 | Baris 21–35

```
    print(f"{kat:<8}", ", ".join(f"{k} ({n:,})" for n, k in top5))

import sqlite3
SIAP = Path("siap_pakai"); SIAP.mkdir(exist_ok=True)
baris = [(kat, kata, n) for (kat, kata), n in gabung.items()]
hasil_df = pd.DataFrame(baris, columns=["kategori", "kata", "jumlah"])
hasil_df.to_parquet(SIAP / "kata_per_kategori.parquet", index=False)

# "sqoop export": pindahkan hasil ke database relasional
db = sqlite3.connect(SIAP / "laporan.db")
hasil_df.to_sql("kata_per_kategori", db, if_exists="replace", index=False)
print("Baris diekspor:", db.execute("SELECT COUNT(*) FROM
kata_per_kategori").fetchone()[0])
print(db.execute("""SELECT kata, jumlah FROM kata_per_kategori
                    WHERE kategori = 'health' ORDER BY jumlah DESC LIMIT
3""").fetchall())
db.close()
```

Gambar 67. Kode H-13 bagian 2 dari 2

Notebook Praktikum 5 | Sel 48 | Hasil eksekusi

```
finance  corona (2,470), rp (1,385), ri (1,125), harga (701), jokowi (642)
food     makanan (699), makan (539), foto (458), galeri (438), enak (358)
health   corona (2,590), virus (1,040), pasien (471), indonesia (427),
meninggal (413)
hot      corona (1,658), anak (793), rumah (561), meninggal (545), the (528)
inet     corona (845), virus (341), ponsel (309), bikin (253), indonesia (249)
news     corona (7,539), polisi (2,244), kasus (2,015), warga (1,972), positif
(1,876)
oto      mobil (1,205), corona (576), motor (432), rp (267), indonesia (256)
sport    corona (228), indonesia (223), motogp (209), masters (127), olimpiade
(124)
travel   corona (1,023), wisata (525), foto (333), dunia (299), turis (284)
Baris diekspor: 73601
[('corona', 2590), ('virus', 1040), ('pasien', 471)]
```

Gambar 68. Hasil H-13

Sebanyak 73.601 pasangan kategori-kata diekspor ke database laporan.db. Kategori health memiliki kata teratas corona 2.590, virus 1.040, dan pasien 471. Folder siap_pakai berisi hasil yang dapat dipakai untuk query laporan.

Tabel 3. Pemeriksaan hasil utama pipeline

Pemeriksaan	Hasil
Baris mentah	91.017
Baris bersih	90.818
File JSONL	19

Pemeriksaan	Hasil
Partisi kategori	9
Pasangan mapper	825.601
Kata unik	32.275
Pasangan setelah combiner	74.009
Hasil pipe sama dengan H-9	True
Baris ekspor kategori-kata	73.601

E. Analisis Hasil

1. Mengapa 2 proses tanpa combiner bisa lebih lambat daripada 1 proses? Apa yang berubah ketika combiner dipasang?

   Pekerjaan tokenisasi dataset ini relatif kecil. Menjalankan proses tambahan memerlukan pembuatan worker, pembacaan split, serialisasi, dan pemindahan hasil antarproses. Tanpa combiner, hasil pengujian menghasilkan 825.601 pasangan yang dikembalikan dari mapper, sesuai acuan modul. Biaya koordinasi dapat melebihi penghematan komputasi, sehingga 2 proses bisa lebih lambat.

   Dengan combiner, mapper menjumlahkan kata secara lokal dan hanya mengirim 74.009 pasangan: berkurang 91,04%, atau sekitar 11,16 kali lebih sedikit. Hasil akhir tetap identik. Pengurangan pasangan tidak menjamin waktu 2 proses pasti lebih cepat; bandingkan median waktu yang muncul ketika H-10 dijalankan di Colab. Tabel di bawah memakai hasil eksekusi terbaru.

Tabel 4. Median waktu pemrosesan hasil eksekusi terbaru

Skenario	Detik	Pasangan
1 proses tanpa combiner	0,116	825.601
1 proses dengan combiner	0,137	74.009
2 proses tanpa combiner	0,207	825.601
2 proses dengan combiner	0,118	74.009

Pada pengujian tanpa combiner, dua proses lebih lambat daripada satu proses. Dengan combiner, dua proses lebih cepat daripada satu proses yang sama-sama memakai combiner. Perbandingan antarskenario harus menyebut kombinasi proses dan

combiner, karena 0,118 detik belum lebih cepat daripada 0,116 detik pada satu proses tanpa combiner.

2. Mengapa menambah reducer dari 4 menjadi 64 tidak membuat beban reducer terberat mendekati bagian yang adil? Bagaimana combiner mengubahnya?

   Semua pasangan dengan key sama wajib masuk ke reducer yang sama. Kata di menghasilkan 27.390 pasangan, sekitar 3,32% dari seluruh pasangan. Pada 64 reducer, bagian adil hanya 1,56%, sehingga satu kata itu sendiri sudah melebihi bagian adil. Reducer terberat menerima 4,28%, sekitar 2,74 kali bagian adil; pada 4 reducer beban terberat 29,17% dibanding 25%.

   Combiner mengirim satu pasangan per kata unik per mapper, sehingga kata sangat sering tidak lagi menghasilkan satu kiriman per kemunculan. Pada 4 reducer, pembagian pasangan menjadi sekitar 24,7%–25,6%. Combiner menekan skew lalu lintas pasangan; ia tidak otomatis menyamakan ukuran input split atau waktu kerja semua mapper.

3. Apakah rp, ri, dan the sebaiknya ditambahkan ke stopword? Jelaskan dari sisi tujuan analisis.

   Tergantung tujuan. Untuk merangkum topik umum, rp dan ri dapat menutupi kata yang lebih informatif, sedangkan the biasanya merupakan kata fungsi bahasa Inggris. Ketiganya masuk akal diuji sebagai stopword domain. Namun, pada analisis nilai uang, ekonomi, atau penyebutan negara, rp dan ri dapat berguna. Bandingkan top kata sebelum/sesudah dan catat alasannya; jangan membuang kata hanya karena frekuensinya tinggi.

4. Seluruh baris inet memiliki URL rusak. Apa dampaknya bila URL dipakai sebagai kunci join? Mengapa baris itu tidak dibuang?

   5.640 URL mentah inet berisi #. Bila dipakai sebagai kunci, URL tersebut tidak mengidentifikasi artikel secara unik; join dengan nilai # yang berulang dapat menghasilkan pasangan many-to-many dan menggandakan baris. Setelah URL dikosongkan, perilaku null juga harus diperhatikan: SQL biasanya tidak mencocokkan null, sedangkan pandas merge dapat mencocokkan kunci null antartabel. Karena itu keluarkan kunci tidak valid dari join atau gunakan ID pengganti yang tercatat.

   Judulnya tetap bermanfaat untuk word count. Membuang seluruh baris inet menghilangkan satu kategori dan menimbulkan bias. Pipeline mempertahankan teksnya,

mengosongkan URL/ID yang rusak, lalu tetap menerapkan deduplikasi serta batas panjang judul. Jumlah inet setelah cleaning menjadi 5.577.

5. Mengapa dipilih partisi kategori dibanding kategori + bulan untuk H-9? Hubungkan dengan jumlah mapper dan small file problem.

   Kategori menghasilkan 9 berkas, sehingga terdapat 9 tugas mapper, sesuai analisis per kategori. Kategori + bulan menghasilkan 54 berkas dengan total data hampir sama. Akibatnya ada lebih banyak pekerjaan kecil, pembukaan file, metadata, dan biaya penjadwalan. Pada dataset 90.818 judul, tambahan overhead tersebut dapat lebih besar daripada manfaat paralelismenya. Partisi kategori juga memungkinkan query finance membaca 1 dari 9 berkas. Pilihan ini tetap memiliki kelemahan: ukuran split news jauh lebih besar daripada sport.

6. Mengapa hasil harus diurutkan sebelum masuk ke reducer.py? Apa yang terjadi bila sort dihilangkan?

   Reducer hanya menyimpan total untuk key yang sedang dibaca. Ia mencetak hasil ketika key berubah, sehingga semua kemunculan key yang sama harus berurutan. Tanpa sort, kata yang muncul lagi setelah kata lain akan dicetak sebagai kelompok baru. Misalnya a, b, a menghasilkan dua baris untuk a, bukan satu total global. Mengubah keluaran seperti itu langsung menjadi dict juga dapat menimpa subtotal sebelumnya. sort menjalankan fungsi pengelompokan shuffle agar hasil pipe sama dengan H-9.

7. Mengapa H-4 memakai sitemap berita, bukan membuka setiap artikel? Apa untung ruginya bagi server dan kode?

   Sitemap berita sudah menyediakan URL, judul, tanggal, dan kadang kata kunci. Beberapa request XML cukup untuk mengambil 20 judul, sehingga lebih ringan bagi server daripada membuka 20 halaman artikel lengkap. Bagi kode, parser tidak bergantung pada layout HTML artikel yang mudah berubah.

   Keterbatasannya: sitemap tidak menyediakan isi artikel lengkap, kata kunci dapat kosong, cakupannya mengikuti entri yang masih dimuat, dan kualitas metadata tanggal perlu diperiksa. Format sitemap atau aturan akses juga dapat berubah. Robots, batas request, jeda 2 detik, serta penghentian saat 403/429 tetap berlaku.

F. Studi Kasus

Analisis 90.818 judul berita detik.com periode 1 Januari–13 Juni 2020 menunjukkan bahwa perhatian terhadap corona memuncak pada Maret. Jumlah judul yang memuat “corona” naik dari 385 pada Januari menjadi 1.077 pada Februari, lalu mencapai 6.994 pada Maret. Setelah itu jumlahnya turun menjadi 4.913 pada April dan 2.934 pada Mei. Juni mencatat 943 judul, tetapi hanya mencakup 13 hari sehingga tidak dapat dibandingkan langsung dengan bulan penuh.

Berdasarkan frekuensi token per kategori, news mencatat kemunculan “corona” terbanyak, yaitu 7.539, disusul health sebanyak 2.590 dan finance sebanyak 2.470. Angka ini menghitung kemunculan kata, bukan otomatis jumlah artikel unik, dan dipengaruhi banyaknya judul dalam tiap kategori. Kategori food memiliki pola lima kata teratas yang paling berbeda secara kualitatif: makanan, makan, foto, galeri, dan enak. Berbeda dari kategori lain, corona tidak masuk lima kata teratasnya.

Temuan tersebut menunjukkan dominasi tema pandemi sekaligus bertahannya topik kuliner. Namun, data hanya berisi judul, bukan isi artikel, sehingga tidak cukup untuk menyimpulkan kedalaman pembahasan atau sentimen. Ringkasan ini menggambarkan dataset yang tersedia, bukan seluruh pemberitaan sepanjang 2020.

G. Latihan

1. Stopword domain

   Soal: Tambahkan 5 kata ke STOPWORD yang menurut Anda mengganggu hasil, lalu jalankan ulang H-13. Catat kata yang masuk dan perubahan lima kata teratas tiap kategori.

   Tambahan yang diuji adalah rp, ri, the, foto, dan galeri. Dua kata pertama merupakan singkatan umum, the kata fungsi Inggris, sedangkan foto dan galeri lebih sering menandai format penyajian. Pilihan ini ditujukan untuk ringkasan topik; kata-kata tersebut belum tentu harus dibuang untuk tujuan lain. STOPWORD dasar dan hasil H-13 dipertahankan agar perbandingan dapat diulang.

Notebook Praktikum 5 | Sel 61 | Kode | Bagian 1/2 | Baris 1–19

```
TAMBAHAN_STOPWORD = {"rp", "ri", "the", "foto", "galeri"}
STOPWORD_DOMAIN = STOPWORD | TAMBAHAN_STOPWORD

def mapper_kategori_domain(path):
    kategori = path.parent.name.split("=", 1)[1]
    hitungan = collections.Counter()
    for judul in pd.read_parquet(path, columns=["judul"])["judul"]:
        for kata in tokenisasi(judul):
            if kata not in STOPWORD_DOMAIN:
                hitungan[(kategori, kata)] += 1
    return hitungan

hasil_domain = collections.Counter()
for hasil_lokal in map(mapper_kategori_domain, splits):
    hasil_domain.update(hasil_lokal)

def top5_kategori(hitungan, kategori):
    return sorted([(n, k) for (kat, k), n in hitungan.items() if kat ==
kategori], reverse=True)[:5]

```

Gambar 69. Kode Latihan 1 bagian 1 dari 2

Notebook Praktikum 5 | Sel 61 | Kode | Bagian 2/2 | Baris 20–37

```
perbandingan_stopword = []
for kategori in sorted(KATEGORI_DIKENAL):
    sebelum = top5_kategori(gabung, kategori)
    sesudah = top5_kategori(hasil_domain, kategori)
    lama = {k for n, k in sebelum}
    baru = {k for n, k in sesudah}
    perbandingan_stopword.append({
        "kategori": kategori,
        "sebelum": ", ".join(f"{k} ({n})" for n, k in sebelum),
        "sesudah": ", ".join(f"{k} ({n})" for n, k in sesudah),
        "keluar": ", ".join(sorted(lama - baru)) or "—",
        "masuk": ", ".join(sorted(baru - lama)) or "—",
    })
tabel_stopword = pd.DataFrame(perbandingan_stopword)
print("Tambahan:", sorted(TAMBAHAN_STOPWORD))
print(tabel_stopword.to_string(index=False))
pd.DataFrame([(kat, kata, n) for (kat, kata), n in hasil_domain.items()],
             columns=["kategori", "kata", "jumlah"]).to_parquet(SIAP /
"latihan1_stopword_domain.parquet", index=False)
```

Gambar 70. Kode Latihan 1 bagian 2 dari 2

Notebook Praktikum 5 | Sel 61 | Hasil eksekusi | Bagian 1/2

```
Tambahan: ['foto', 'galeri', 'ri', 'rp', 'the']
kategori
sebelum
sesudah       keluar        masuk
 finance              corona (2470), rp (1385), ri (1125), harga (701), jokowi
(642)          corona (2470), harga (701), jokowi (642), mau (562), ekonomi
(536)       ri, rp ekonomi, mau
    food            makanan (699), makan (539), foto (458), galeri (438), enak
(358)            makanan (699), makan (539), enak (358), bikin (343), rumah
(311) foto, galeri bikin, rumah
  health corona (2590), virus (1040), pasien (471), indonesia (427), meninggal
(413) corona (2590), virus (1040), pasien (471), indonesia (427), meninggal
(413)            –           –
     hot          corona (1658), anak (793), rumah (561), meninggal (545), the
(528)         corona (1658), anak (793), rumah (561), meninggal (545), film
(489)          the         film
    inet       corona (845), virus (341), ponsel (309), bikin (253), indonesia
(249)       corona (845), virus (341), ponsel (309), bikin (253), indonesia
(249)            –           –
```

Gambar 71. Hasil Latihan 1 bagian 1 dari 2

Notebook Praktikum 5 | Sel 61 | Hasil eksekusi | Bagian 2/2

```
    news    corona (7539), polisi (2244), kasus (2015), warga (1972), positif
(1876)    corona (7539), polisi (2244), kasus (2015), warga (1972), positif
(1876)            –           –
     oto          mobil (1205), corona (576), motor (432), rp (267), indonesia
(256)       mobil (1205), corona (576), motor (432), indonesia (256), honda
(255)           rp        honda
   sport corona (228), indonesia (223), motogp (209), masters (127), olimpiade
(124) corona (228), indonesia (223), motogp (209), masters (127), olimpiade
(124)            –           –
  travel           corona (1023), wisata (525), foto (333), dunia (299), turis
(284)      corona (1023), wisata (525), dunia (299), turis (284), indonesia
(269)         foto    indonesia
```

Gambar 72. Hasil Latihan 1 bagian 2 dari 2

Hasil perubahan: finance mengeluarkan ri/rp dan memasukkan ekonomi/mau; food mengeluarkan foto/galeri dan memasukkan bikin/rumah; hot mengganti the dengan film; oto mengganti rp dengan honda; travel mengganti foto dengan indonesia. Top-5 health, inet, news, dan sport tetap. Tabel output di atas mencatat frekuensi sebelum dan sesudah untuk seluruh kategori.

2. Frekuensi kata per bulan

Soal: Ubah key di H-13 menjadi (bulan, kata). Kata apa yang berubah dari Januari ke Mei?

Notebook Praktikum 5 | Sel 64 | Kode | Bagian 1/2 | Baris 1–18

```
def mapper_bulan(path):
    hitungan = collections.Counter()
    frame = pd.read_parquet(path, columns=["bulan", "judul"])
    for bulan, judul in frame.itertuples(index=False, name=None):
        for kata in tokenisasi(judul):
            if kata not in STOPWORD:
                hitungan[(bulan, kata)] += 1
    return hitungan

hasil_bulan = collections.Counter()
for hitungan in map(mapper_bulan, splits):
    hasil_bulan.update(hitungan)

puncak_bulanan = {}
for bulan in sorted(bersih["bulan"].unique()):
    puncak_bulanan[bulan] = sorted([(n, k) for (b, k), n in
hasil_bulan.items() if b == bulan], reverse=True)[:5]
    print(bulan, ", ".join(f"{k} ({n:,})" for n, k in
puncak_bulanan[bulan]))

```

Gambar 73. Kode Latihan 2 bagian 1 dari 2

Notebook Praktikum 5 | Sel 64 | Kode | Bagian 2/2 | Baris 19–29

```
jan = {k for n, k in puncak_bulanan["2020-01"]}
mei = {k for n, k in puncak_bulanan["2020-05"]}
print("\nKeluar dari top-5 Januari:", sorted(jan - mei))
print("Masuk top-5 Mei:", sorted(mei - jan))
print("Tetap di top-5:", sorted(jan & mei))
print("\nFrekuensi token corona dan proporsi judul:")
for bulan in ("2020-01", "2020-05"):
    bagian = bersih[bersih["bulan"] == bulan]
    judul_corona = bagian["judul"].str.lower().str.contains("corona",
regex=False).sum()
    print(f"{bulan}: token corona={hasil_bulan[(bulan, 'corona')]:,}; "
          f"judul memuat corona={judul_corona:,}/{len(bagian):,}
({judul_corona/len(bagian)*100:.2f}%)")
```

Gambar 74. Kode Latihan 2 bagian 2 dari 2

Notebook Praktikum 5 | Sel 64 | Hasil eksekusi

```
2020-01 banjir (1,220), rp (616), polisi (554), jokowi (516), tahun (493)
2020-02 corona (1,076), polisi (630), virus (596), rp (572), banjir (572)
2020-03 corona (6,986), virus (1,340), positif (925), pasien (749), rumah
(695)
2020-04 corona (4,914), positif (828), pandemi (706), psbb (689), warga (609)
2020-05 corona (2,935), kasus (640), positif (628), pandemi (596), warga (565)
2020-06 corona (943), normal (471), new (465), kasus (315), rp (265)

Keluar dari top-5 Januari: ['banjir', 'jokowi', 'polisi', 'rp', 'tahun']
Masuk top-5 Mei: ['corona', 'kasus', 'pandemi', 'positif', 'warga']
Tetap di top-5: []

Frekuensi token corona dan proporsi judul:
2020-01: token corona=385; judul memuat corona=385/16,258 (2.37%)
2020-05: token corona=2,935; judul memuat corona=2,934/16,352 (17.94%)
```

Gambar 75. Hasil Latihan 2

Pembahasan: seluruh anggota top-5 Januari keluar dan digantikan lima kata bertema pandemi pada Mei. Peringkat menggambarkan pergeseran kosakata judul. Jumlah judul yang memuat corona meningkat dari 385 pada Januari menjadi 2.934 pada Mei. Frekuensi token dan jumlah judul harus dibedakan karena satu judul dapat memuat token yang sama lebih dari sekali; proporsi terhadap semua judul per bulan membantu perbandingan ketika volume berita berbeda.

Top-5 Januari adalah banjir (1.220), rp (616), polisi (554), jokowi (516), dan tahun (493). Pada Mei berubah menjadi corona (2.935), kasus (640), positif (628), pandemi (596), dan warga (565). Seluruh anggota top-5 berganti, dari topik banjir dan berita umum menuju pandemi. Ini perbandingan frekuensi token; angka judul yang memuat corona pada Mei adalah 2.934.

3. Bigram

Soal: Ubah mapper agar menghasilkan pasangan kata berurutan, misalnya virus corona. Berapa pasangan yang dihasilkan, dan berapa setelah combiner?

Notebook Praktikum 5 | Sel 66 | Kode | Bagian 1/2 | Baris 1–20

```
def mapper_bigram(path):
    pasangan = []
    for judul in pd.read_parquet(path, columns=["judul"])["judul"]:
        kata = tokenisasi(judul)
        pasangan.extend((f"{a} {b}", 1) for a, b in zip(kata, kata[1:]))
    return pasangan

def mapper_bigram_combiner(path):
    hitungan = collections.Counter()
    for judul in pd.read_parquet(path, columns=["judul"])["judul"]:
        kata = tokenisasi(judul)
        hitungan.update(f"{a} {b}" for a, b in zip(kata, kata[1:]))
    return hitungan

bigram_tanpa = [mapper_bigram(p) for p in splits]
bigram_dengan = [mapper_bigram_combiner(p) for p in splits]
jumlah_bigram = sum(len(h) for h in bigram_tanpa)
jumlah_bigram_combiner = sum(len(h) for h in bigram_dengan)
reducer_bigram = collections.Counter()
for hasil_lokal in bigram_dengan:
```

Gambar 76. Kode Latihan 3 bagian 1 dari 2

Notebook Praktikum 5 | Sel 66 | Kode | Bagian 2/2 | Baris 21–36

```
    reducer_bigram.update(hasil_lokal)
cek_bigram = collections.Counter()
for pasangan in bigram_tanpa:
    for k, n in pasangan:
        cek_bigram[k] += n
assert cek_bigram == reducer_bigram
rumus_bigram = sum(max(len(tokenisasi(j)) - 1, 0) for j in bersih["judul"])
assert jumlah_bigram == rumus_bigram
print(f"Pasangan tanpa combiner : {jumlah_bigram:,}")
print(f"Pasangan dengan combiner: {jumlah_bigram_combiner:,}")
print(f"Bigram unik global      : {len(reducer_bigram):,}")
print(f"Pengurangan transfer    : {(1-
jumlah_bigram_combiner/jumlah_bigram)*100:.2f}%")
print("Hasil akhir sama        :", cek_bigram == reducer_bigram)
print("\n10 bigram teratas:")
for kata, n in reducer_bigram.most_common(10):
    print(f"{kata:<30} {n:>8,}")
```

Gambar 77. Kode Latihan 3 bagian 2 dari 2

Notebook Praktikum 5 | Sel 66 | Hasil eksekusi

```
Pasangan tanpa combiner : 734,783
Pasangan dengan combiner: 411,217
Bigram unik global      : 349,114
Pengurangan transfer    : 44.04%
Hasil akhir sama        : True

10 bigram teratas:
virus corona                    2,915
corona di                       2,178
positif corona                  1,804
di tengah                       1,258
pandemi corona                  1,075
di rumah                        1,047
new normal                        917
di indonesia                      871
rp juta                           700
hari ini                          627
```

Gambar 78. Hasil Latihan 3

Jawaban hasil eksekusi: terdapat 734.783 pasangan bigram tanpa combiner dan 411.217 pasangan setelah combiner, dengan 349.114 bigram unik global. Pengurangan transfer sekitar 44,04% dan hasil reduce kedua cara sama.

Pembahasan: satu judul dengan n token menghasilkan maksimum n−1 bigram. Jumlah pasangan setelah combiner adalah penjumlahan bigram unik per split, sehingga umumnya lebih besar daripada jumlah bigram unik global. Combiner mengurangi data yang dikirim, sementara frekuensi akhirnya tetap identik.

4. Jumlah reducer

Soal: Jalankan H-11 dengan R = 2, 8, dan 32. Buat tabel beban reducer terberat dibanding bagian yang adil.

Notebook Praktikum 5 | Sel 68 | Kode | Baris 1–16

```
baris_reducer = []
for r in (2, 8, 32):
    b_tanpa = beban(hasil_map, r)
    b_dengan = beban([list(c.items()) for c in h2], r)
    adil = 100 / r
    terberat = max(b_tanpa) / sum(b_tanpa) * 100
    baris_reducer.append({
        "R": r,
        "pasangan reducer terberat": max(b_tanpa),
        "terberat tanpa (%)": terberat,
        "bagian adil (%)": adil,
        "rasio terhadap adil": terberat / adil,
        "terberat combiner (%)": max(b_dengan) / sum(b_dengan) * 100,
    })
tabel_reducer = pd.DataFrame(baris_reducer)
print(tabel_reducer.to_string(index=False, float_format=lambda x: f"
{x:.3f}"))
```

Gambar 79. Kode Latihan 4

Notebook Praktikum 5 | Sel 68 | Hasil eksekusi

```
 R  pasangan reducer terberat  terberat tanpa (%)  bagian adil (%)  rasio
terhadap adil  terberat combiner (%)
 2                    431563             52.273           50.000
1.045                50.234
 8                    138978             16.834           12.500
1.347                12.852
32                     48073              5.823            3.125
1.863                 3.439
```

Gambar 80. Hasil Latihan 4

Pada R = 2, 8, dan 32, rasio beban terberat terhadap bagian adil masing-masing sekitar 1,045; 1,347; dan 1,863. Nilai lengkap ditampilkan pada tabel berikut.

Tabel 5. Beban reducer terberat

R	Pasangan terberat	Beban persen	Adil persen	Rasio
2	431.563	52,273	50,000	1,045
8	138.978	16,834	12,500	1,347
32	48.073	5,823	3,125	1,863

Pembahasan: bandingkan rasio terhadap bagian adil, bukan persentase terberat saja. Saat jumlah reducer naik, persentase terberat bisa turun tetapi ketimpangan relatif meningkat karena satu key tetap tidak dapat dibagi ke banyak reducer. Tabel menghitung beban pasangan transfer; ia bukan pengukuran waktu reducer pada cluster.

5. Crawl ulang

Soal: Tambahkan jeda 0,05 detik pada crawl_iter untuk 50 halaman pertama. Berapa waktunya, dan berapa lama bila 911 halaman memakai jeda 1 detik?

Notebook Praktikum 5 | Sel 70 | Kode | Baris 1–13

```
t0_latihan5 = time.perf_counter()
hasil_50 = list(crawl_iter(BASE, range(1, 51), delay=0.05))
waktu_50 = time.perf_counter() - t0_latihan5
waktu_nonjeda_50 = max(0.0, waktu_50 - 50 * 0.05)
estimasi_nonjeda_911 = waktu_nonjeda_50 / 50 * N_HALAMAN
estimasi_911 = N_HALAMAN * 1.0 + estimasi_nonjeda_911
print("Halaman diuji          :", 50)
print("Judul terkumpul        :", len(hasil_50))
print(f"Waktu aktual 50 halaman : {waktu_50:.3f} detik")
print(f"Jeda minimum 50 halaman : {50*0.05:.1f} detik")
print(f"Jeda saja 911 halaman   : {N_HALAMAN:.0f} detik = {N_HALAMAN/60:.2f}
menit")
print(f"Estimasi total 911     : {estimasi_911:.2f} detik =
{estimasi_911/60:.2f} menit")
assert len(hasil_50) == 5000
```

Gambar 81. Kode Latihan 5

Notebook Praktikum 5 | Sel 70 | Hasil eksekusi

```
Halaman diuji          : 50
Judul terkumpul        : 5000
Waktu aktual 50 halaman : 3.480 detik
Jeda minimum 50 halaman : 2.5 detik
Jeda saja 911 halaman   : 911 detik = 15.18 menit
Estimasi total 911     : 928.86 detik = 15.48 menit
```

Gambar 82. Hasil Latihan 5

Jawaban hasil eksekusi: 50 halaman (5.000 judul) dengan jeda 0,05 detik memerlukan 3,480 detik. Estimasi 911 halaman dengan jeda 1 detik adalah 928,86 detik, sekitar 15 menit 28,9 detik, termasuk overhead yang diekstrapolasi dari pengujian 50 halaman.

Pembahasan: 911 detik sama dengan 15 menit 11 detik untuk jeda saja. Waktu total lebih lama karena request lokal, pembuatan HTML, parsing, dan kemungkinan retry. Estimasi di atas mengekstrapolasi overhead dari 50 halaman; pengujian 911 halaman dengan jeda 1 detik tidak dijalankan, sehingga angka estimasi bukan waktu aktual.

6. Sitemap lain

Soal: Ganti satu alamat di SITEMAP_DIPILIH dengan sitemap_news dari bagian lain yang tercantum di sitemap.xml. Kategori apa yang muncul? Terapkan aturan_kualitas dari H-6 pada 20 judul hasil crawl dan laporkan hasilnya.

Notebook Praktikum 5 | Sel 72 | Kode | Bagian 1/4 | Baris 1–18

```
sampel_latihan6 = []
status_latihan6 = {}
SITEMAP_LATIHAN6 = list(SITEMAP_DIPILIH)
try:
    sesi_latihan6 = requests.Session()
    sesi_latihan6.headers["User-Agent"] = UA_DETIK
    INDEX_SITEMAP = "https://www.detik.com/sitemap.xml"
    isi_indeks = ambil_xml(INDEX_SITEMAP, sesi_latihan6)
    sup_indeks = BeautifulSoup(isi_indeks, "xml")
    alamat_indeks = [x.get_text(strip=True) for x in
sup_indeks.find_all("loc")]
    calon = [u for u in alamat_indeks
             if urlparse(u).path.endswith("sitemap_news.xml")
             and u not in SITEMAP_DIPILIH
             and urlparse(u).hostname
             and urlparse(u).hostname.endswith(".detik.com")
             and urlparse(u).hostname.split(".")[0] in KATEGORI_DIKENAL]
    if not calon:
        raise ValueError("Tidak ada sitemap berita lain yang sesuai pada
indeks saat ini.")
```

Gambar 83. Kode Latihan 6 bagian 1 dari 4

Notebook Praktikum 5 | Sel 72 | Kode | Bagian 2/4 | Baris 19–33

```
    # Prioritaskan kategori yang belum dipakai di H-4.
    kategori_awal = {urlparse(u).netloc.split(".")[0] for u in
SITEMAP_DIPILIH}
    calon.sort(key=lambda u: (urlparse(u).netloc.split(".")[0] in
kategori_awal, u))
    sitemap_baru = calon[0]
    SITEMAP_LATIHAN6[2] = sitemap_baru
    print("Sitemap awal :", SITEMAP_DIPILIH)
    print("Sitemap baru :", SITEMAP_LATIHAN6)
    for posisi, alamat in enumerate(SITEMAP_LATIHAN6):
        kategori_url = urlparse(alamat).netloc.split(".")[0]
        cache = [x for x in sampel if x["kategori"] == kategori_url] if
posisi < 2 else []
        entri = cache[:BATAS_PER_BAGIAN] if len(cache) >= BATAS_PER_BAGIAN
else parse_sitemap_news(ambil_xml(alamat, sesi_latihan6))[:BATAS_PER_BAGIAN]
        sisa = BATAS_TOTAL - len(sampel_latihan6)
        sampel_latihan6.extend(entri[:sisa])
        if len(sampel_latihan6) >= BATAS_TOTAL:
            break
```

Gambar 84. Kode Latihan 6 bagian 2 dari 4

Notebook Praktikum 5 | Sel 72 | Kode | Bagian 3/4 | Baris 34–47

```
    if not sampel_latihan6:
        raise ValueError("Sitemap tidak mengandung entri berita yang dapat
diparsing.")
    frame_latihan6 = pd.DataFrame(sampel_latihan6)
    print("\nJumlah judul:", len(frame_latihan6))
    print("Kategori:", frame_latihan6["kategori"].value_counts().to_dict())
    kualitas_latihan6 = pd.DataFrame([
        {"aturan": nama, "lulus": int(lulus.sum()), "gagal":
int((~lulus).sum())}
        for nama, lulus in aturan_kualitas(frame_latihan6).items()
    ])
    print(kualitas_latihan6.to_string(index=False))
    if len(frame_latihan6) != 20:
        print("Catatan: entri tersedia kurang dari target 20; jumlah aktual
dilaporkan apa adanya.")
    Path("contoh_crawl").mkdir(exist_ok=True)
    frame_latihan6.to_json("contoh_crawl/latihan6_sitemap_lain.jsonl",
orient="records", lines=True, force_ascii=False)
```

Gambar 85. Kode Latihan 6 bagian 3 dari 4

Notebook Praktikum 5 | Sel 72 | Kode | Bagian 4/4 | Baris 48–54

```
    status_latihan6 = {"berhasil": True, "jumlah": len(frame_latihan6),
"sitemap_baru": sitemap_baru,
                       "kategori":
frame_latihan6["kategori"].value_counts().to_dict()}
except Exception as e:
    sampel_latihan6 = []  # Jangan memakai hasil parsial sebagai hasil
target lengkap.
    status_latihan6 = {"berhasil": False, "error": f"{type(e).__name__}:
{e}"}
    print("Latihan 6 belum menghasilkan sampel:", status_latihan6["error"])
print("Status aktual:", status_latihan6)
```

Gambar 86. Kode Latihan 6 bagian 4 dari 4

Notebook Praktikum 5 | Sel 72 | Hasil eksekusi

```
Sitemap awal : ['https://news.detik.com/berita/sitemap_news.xml',
'https://inet.detik.com/news/sitemap_news.xml',
'https://sport.detik.com/basket/sitemap_news.xml']
Sitemap baru : ['https://news.detik.com/berita/sitemap_news.xml',
'https://inet.detik.com/news/sitemap_news.xml',
'https://finance.detik.com/berita-ekonomi-bisnis/sitemap_news.xml']

Jumlah judul: 20
Kategori: {'news': 7, 'inet': 7, 'finance': 6}
              aturan  lulus  gagal
  judul tidak kosong     20      0
     judul 3-30 kata     20      0
    kategori dikenal     20      0
       tanggal valid     20      0
   url artikel valid     20      0
judul tidak duplikat     20      0
Status aktual: {'berhasil': True, 'jumlah': 20, 'sitemap_baru':
'https://finance.detik.com/berita-ekonomi-bisnis/sitemap_news.xml',
'kategori': {'news': 7, 'inet': 7, 'finance': 6}}
```

Gambar 87. Hasil Latihan 6

Jawaban hasil eksekusi: sitemap basket diganti dengan https://finance.detik.com/berita-ekonomi-bisnis/sitemap_news.xml, yang ditemukan dalam indeks. Terkumpul 20 judul: news 7, inet 7, finance 6. Hasil aturan kualitas: judul tidak kosong: 20 lulus, 0 gagal; judul 3-30 kata: 20 lulus, 0 gagal; kategori dikenal: 20 lulus, 0 gagal; tanggal valid: 20 lulus, 0 gagal; url artikel valid: 20 lulus, 0 gagal; judul tidak duplikat: 20 lulus, 0 gagal.

Pembahasan: kategori dan kegagalan aturan kualitas dilaporkan dari data aktual pada output sel. Sitemap baru belum tentu menghasilkan kategori baru: beberapa bagian bisa memakai subdomain kategori yang sama. Aturan 3–30 kata berasal dari dataset

historis; judul terbaru yang melanggarnya harus dicatat, bukan dipaksa lolos. Jika akses gagal, bagian empiris ini memerlukan eksekusi ulang saat situs mengizinkan akses.

7. Kata kunci

Soal: Kolom kata_kunci berisi kata kunci tiap berita yang dipisah koma. Tulis mapper dan reducer untuk menghitung kata kunci terbanyak pada hasil crawl Anda.

Notebook Praktikum 5 | Sel 74 | Kode | Bagian 1/2 | Baris 1–19

```
def mapper_kata_kunci(entri):
    for berita in entri:
        isi = berita.get("kata_kunci") or ""
        for kunci in isi.split(","):
            kunci = re.sub(r"\s+", " ", kunci.strip().lower())
            if kunci:
                yield kunci, 1

def reducer_kata_kunci(kunci, nilai):
    return kunci, sum(nilai)

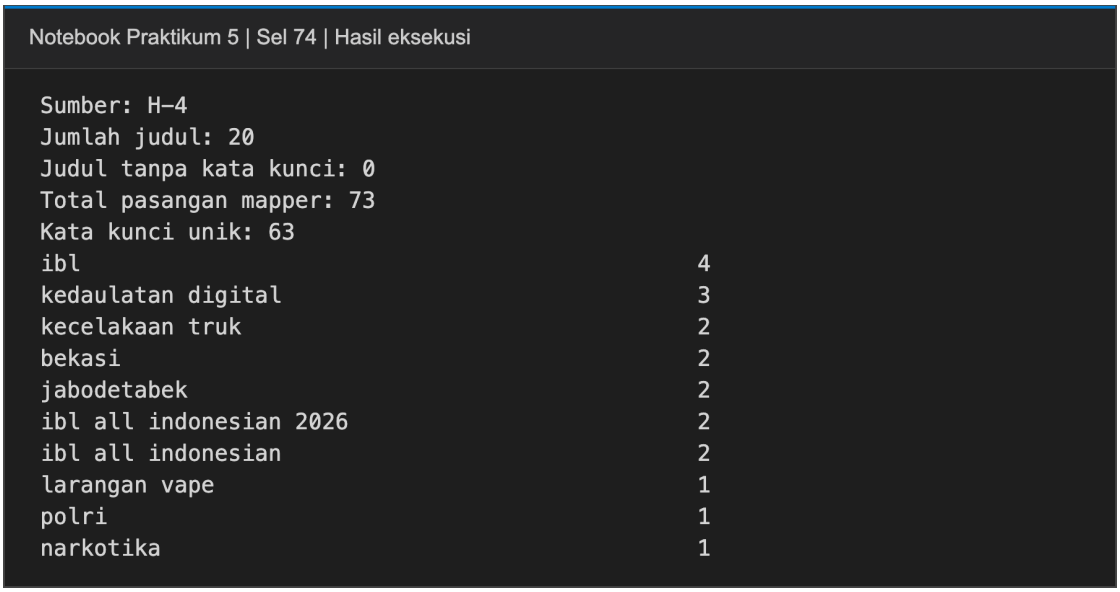
sumber_kata_kunci = sampel if sampel else sampel_latihan6
hasil_kata_kunci = {}
if sumber_kata_kunci:
    shuffle_kunci = collections.defaultdict(list)
    for kunci, nilai in mapper_kata_kunci(sumber_kata_kunci):
        shuffle_kunci[kunci].append(nilai)
    hasil_kata_kunci = dict(reducer_kata_kunci(k, v) for k, v in
shuffle_kunci.items())
    print("Sumber:", "H-4" if sampel else "Latihan 6")
```

Gambar 88. Kode Latihan 7 bagian 1 dari 2

Notebook Praktikum 5 | Sel 74 | Kode | Bagian 2/2 | Baris 20–30

```
    print("Jumlah judul:", len(sumber_kata_kunci))
    print("Judul tanpa kata kunci:", sum(not (b.get("kata_kunci") or
"").strip() for b in sumber_kata_kunci))
    print("Total pasangan mapper:", sum(map(len, shuffle_kunci.values())))
    print("Kata kunci unik:", len(hasil_kata_kunci))
    for kunci, frekuensi in
collections.Counter(hasil_kata_kunci).most_common(10):
        print(f"{kunci:<45} {frekuensi:>4}")
    if not hasil_kata_kunci:
        print("Tidak ada metadata kata kunci terisi; frekuensi tidak dapat
ditentukan.")
else:
    print("Belum dapat menghitung kata kunci: tidak ada hasil crawl langsung
yang berhasil.")
    print("Jalankan kembali H-4 atau Latihan 6 saat akses situs tersedia,
lalu jalankan sel ini.")
```

Gambar 89. Kode Latihan 7 bagian 2 dari 2



Gambar 90. Hasil Latihan 7

Jawaban hasil eksekusi: 20 judul crawl menghasilkan 73 pasangan mapper dan 63 kata kunci unik. Judul tanpa metadata kata kunci: 0. Sepuluh kata kunci teratas: ibl (4), kedaulatan digital (3), bekasi (2), ibl all indonesian (2), ibl all indonesian 2026 (2), jabodetabek (2), kecelakaan truk (2), ai (1), aksara jawa (1), andromeda (1). Urutan antarkata dengan frekuensi sama pada ringkasan ini diurutkan secara alfabetis; frekuensinya sama dengan output kode.

Pembahasan: frekuensi di sini menghitung kemunculan label dalam metadata, bukan token penyusun judul. Frasa tidak dipecah menjadi kata tunggal. Metadata kosong tidak membuktikan bahwa suatu topik tidak dibahas; hasil hanya mewakili label yang tersedia pada sampel kecil saat crawl.

H. Tugas Mandiri

1. Inverted index dengan MapReduce

Tugas mandiri membangun indeks terbalik yang memetakan setiap kata ke daftar ID judul yang memuatnya. Mapper menghasilkan pasangan kata-ID, shuffle mengumpulkan semua ID untuk kata yang sama, dan reducer membentuk daftar ID unik. Token yang berulang dalam satu judul hanya menghasilkan satu pasangan kata-ID karena indeks menyimpan keanggotaan dokumen, bukan frekuensi token.

Sebanyak 5.577 ID inet kosong setelah cleaning. Baris tersebut tetap diindeks dengan ID lokal deterministik berupa SHA-256 dari kategori, tanggal, dan judul. ID lokal diberi awalan lokal, sedangkan ID asli diberi awalan detik agar kedua ruang ID

tidak bertabrakan. ID asli yang berulang juga akan memakai jalur pengganti; pada data ini tidak ditemukan ID sumber berulang. ID lokal adalah identitas dokumen untuk indeks, bukan ID artikel asli detik.

Tugas Mandiri | Inverted index | Kode | Bagian 1/3 | Baris 1–19

```
import pandas as pd
import re,hashlib,collections,json
from pathlib import Path

berita_indeks = pd.read_parquet('work/rerun/lapisan_terkurasi')
id_asli = berita_indeks['id'].astype('Int64')
id_berulang = id_asli.notna() & id_asli.duplicated(keep=False)
def buat_id_dokumen(baris, perlu_pengganti):
    if pd.notna(baris['id']) and not perlu_pengganti:
        return f"detik:{int(baris['id'])}"
    isi = f"{baris['kategori']}|{baris['tanggal'].date()}|{baris['judul']}"
    return f"lokal:{hashlib.sha256(isi.encode('utf-8')).hexdigest()}"

berita_indeks['id_dokumen'] = [
    buat_id_dokumen(baris, bool(ulang))
    for (_, baris), ulang in zip(berita_indeks.iterrows(), id_berulang)
]
assert berita_indeks['id_dokumen'].is_unique

```

Gambar 91. Kode Tugas Mandiri inverted index bagian 1 dari 3

Tugas Mandiri | Inverted index | Kode | Bagian 2/3 | Baris 20–36

```
def mapper_indeks(frame):
    for judul, identitas in frame[['judul',
'id_dokumen']].itertuples(index=False, name=None):
        for kata in sorted(set(re.findall(r'[a-z]+', judul.lower()))):
            yield kata, identitas

def reducer_indeks(kata, daftar_id):
    return kata, sorted(set(daftar_id))

shuffle_indeks = collections.defaultdict(list)
for kata, identitas in mapper_indeks(berita_indeks):
    shuffle_indeks[kata].append(identitas)
inverted_index = dict(reducer_indeks(kata, ids) for kata, ids in
shuffle_indeks.items())

print('Dokumen terindeks :', len(berita_indeks))
print('ID sumber kosong  :', int(id_asli.isna().sum()))
print('ID sumber berulang:', int(id_berulang.sum()))
print('ID pengganti lokal:',
int(berita_indeks['id_dokumen'].str.startswith('lokal:').sum()))
```

Gambar 92. Kode Tugas Mandiri inverted index bagian 2 dari 3

Tugas Mandiri | Inverted index | Kode | Bagian 3/3 | Baris 37–45

```
print('Kata unik         :', len(inverted_index))
print('Pasangan kata-ID  :', sum(map(len, inverted_index.values())))
for kata in ['corona', 'banjir', 'teknologi']:
    print(f'\n{kata}: {len(inverted_index[kata])} dokumen')
    print('Lima ID pertama:', ', '.join(inverted_index[kata][:5]))
    doc_ids = set(berita_indeks.loc[berita_indeks['judul'].str.lower().map(
        lambda j: kata in re.findall(r'[a-z]+', j)), 'id_dokumen'])
    assert set(inverted_index[kata]) == doc_ids
print('\nCek silang tiga kata: True')
```

Gambar 93. Kode Tugas Mandiri inverted index bagian 3 dari 3

Tugas Mandiri | Inverted index | Hasil eksekusi

```
Dokumen terindeks : 90818
ID sumber kosong  : 5577
ID sumber berulang: 0
ID pengganti lokal: 5577
Kata unik         : 32275
Pasangan kata-ID  : 817993

corona: 17228 dokumen
Lima ID pertama: detik:4856495, detik:4856598, detik:4868317, detik:4868545,
detik:4869163

banjir: 1992 dokumen
Lima ID pertama: detik:4842198, detik:4842285, detik:4842286, detik:4842287,
detik:4842292

teknologi: 126 dokumen
Lima ID pertama: detik:4853400, detik:4855174, detik:4855582, detik:4857564,
detik:4857763

Cek silang tiga kata: True
```

Gambar 94. Hasil Tugas Mandiri inverted index

Indeks mencakup 90.818 dokumen, 32.275 kata unik, dan 817.993 pasangan kata-ID. Jumlah pasangan ini lebih kecil daripada 825.601 token word count karena kata yang berulang pada judul yang sama hanya dicatat sekali. Cek silang tiga kata contoh terhadap pencarian langsung pada judul menghasilkan True.

Tabel 6. Tiga contoh inverted index dan lima ID pertama

Kata	Jumlah dokumen	Lima ID pertama
corona	17.228	detik:4856495, detik:4856598, detik:4868317, detik:4868545, detik:4869163

Kata	Jumlah dokumen	Lima ID pertama
banjir	1.992	detik:4842198, detik:4842285, detik:4842286, detik:4842287, detik:4842292
teknologi	126	detik:4853400, detik:4855174, detik:4855582, detik:4857564, detik:4857763

I. Pertanyaan Evaluasi

1. Jelaskan tiga tahap MapReduce dengan contoh kasus kantin atau word count. Pada tahap mana data berpindah antarmesin?

   Map menghasilkan pasangan kata dan angka satu untuk setiap token judul. Shuffle mengumpulkan seluruh pasangan dengan kata sama ke reducer tujuan. Reduce menjumlahkan angka tersebut menjadi frekuensi global. Pada cluster, perpindahan data antarmesin terutama terjadi saat shuffle. Pada praktikum ini alurnya disimulasikan pada satu mesin melalui struktur Python dan pipe.

2. Apa fungsi combiner, dan mengapa hasil akhir tidak berubah? Kapan combiner tidak boleh dipakai?

   Combiner menggabungkan nilai lokal sebelum shuffle sehingga jumlah pasangan berkurang. Penjumlahan word count bersifat asosiatif dan komutatif, sehingga pengelompokan penjumlahan tidak mengubah total akhir. Combiner tidak boleh diterapkan secara naif pada operasi yang hasilnya bergantung urutan atau pengelompokan, misalnya rata-rata dari rata-rata kelompok dengan ukuran berbeda. Untuk rata-rata, kirim pasangan jumlah dan banyak data, lalu hitung hasil akhirnya di reducer.

3. Apa yang dimaksud input split, dan bagaimana pembagian partisi di Praktikum 3 memengaruhi jumlah mapper?

   Input split merupakan unit masukan yang diproses sebuah tugas mapper. Praktikum ini memperlakukan satu berkas Parquet sebagai satu split, sehingga sembilan berkas kategori menghasilkan sembilan tugas mapper. Jika dipartisi kategori dan bulan, ada 54 berkas serta 54 tugas mapper. Jumlah tugas tidak sama dengan jumlah proses yang aktif bersamaan; Pool mengatur berapa tugas dapat berjalan paralel.

4. Apa itu data skew? Berikan contoh dari praktikum ini dan dua cara mengurangi dampaknya.

   Data skew adalah pembagian beban yang tidak merata. Kata di muncul 27.390 kali dan semua pasangannya menuju satu reducer, sehingga satu key dapat melampaui bagian adil reducer. Cara pertama adalah memakai combiner untuk mengurangi pasangan sebelum shuffle. Cara kedua adalah memecah hot key dengan salting, menghitung subtotal pada beberapa reducer, lalu menggabungkan subtotal tersebut pada tahap kedua. Menambah reducer saja tidak membagi satu key.

5. Mengapa Hadoop Streaming memungkinkan mapper ditulis dalam Python? Apa kelebihan dan kekurangannya dibanding mapper Java?

   Hadoop Streaming berkomunikasi dengan proses mapper dan reducer melalui stdin serta stdout, sehingga program Python dapat mengikuti protokol teks yang sama. Python memudahkan pengembangan dan pemakaian pustaka analisis. Kekurangannya mencakup biaya startup interpreter, serialisasi teks, dan kemungkinan throughput lebih rendah dibanding implementasi Java yang terintegrasi langsung dengan API Hadoop. Hasil akhirnya tetap harus diuji setara.

6. Apa peran source, channel, dan sink pada Flume? Mengapa channel membatasi ukuran antreannya?

   Source menerima atau mengambil event, channel menampung event sementara, dan sink menulisnya ke tujuan. Pada simulasi ini crawler menjadi source, queue.Queue menjadi channel, dan penulis JSONL menjadi sink. Kapasitas 2.000 event membatasi memori; source menunggu ketika antrean penuh sampai sink mengambil data. Mekanisme backpressure ini mencegah pertumbuhan antrean tanpa batas.

7. Apa yang dilakukan Sqoop import dan export? Mengapa import memakai beberapa mapper dengan rentang kunci?

   Sqoop import memindahkan data dari database relasional ke ekosistem Hadoop, sedangkan export mengirim hasil Hadoop kembali ke database relasional. Rentang kunci membagi pembacaan tabel menjadi beberapa pekerjaan paralel dengan bagian yang dapat dipisahkan. Pemilihan kolom pembagi perlu mempertimbangkan distribusi agar beban tidak timpang. H-13 hanya mensimulasikan arah export menggunakan pandas to_sql ke SQLite, bukan menjalankan Sqoop asli.

8. Sebutkan peran HDFS dan YARN. Di bagian mana peran keduanya terlihat, dan di bagian mana hanya disimulasikan?

   HDFS menyimpan berkas dalam blok yang tersebar pada mesin cluster. YARN mengatur sumber daya dan menjalankan pekerjaan pada cluster. Folder serta partisi Parquet di praktikum menggambarkan pembagian input, tetapi tidak menjalankan HDFS. Pool dan pembagian mapper/reducer menggambarkan pekerjaan paralel tanpa scheduler YARN. Pipe H-12 juga berjalan lokal. Bahkan opsi Hadoop pada modul memakai local mode sehingga tidak mengaktifkan HDFS atau YARN.

J. Kesimpulan

Praktikum menghasilkan pipeline lengkap dari crawling portal sampai penyimpanan hasil per kategori ke SQLite. Pemeriksaan kualitas mempertahankan teks inet yang masih berguna meskipun URL-nya rusak, dan cleaning mengurangi 91.017 baris menjadi 90.818 judul. Partisi kategori menghasilkan sembilan input split serta mendukung partition pruning.

Combiner mengurangi pasangan dari 825.601 menjadi 74.009 dengan hasil identik. Pengujian menunjukkan bahwa proses tambahan belum tentu mempercepat pekerjaan kecil, dan satu hot key tetap dapat membebani reducer walaupun jumlah reducer ditambah. Hasil pipe sama dengan MapReduce Python. Studi kasus menemukan puncak judul corona pada Maret 2020, sedangkan inverted index melengkapi analisis dengan pencarian dokumen berdasarkan kata dan penanganan ID yang kosong.

Link:

Google Drive

Github